Netflix L5 interview preparation

Go engineering and hiring-manager conversations

The reading edition
Worked examples and visual explanations

Data structures. Algorithms. Practical Go programs.
Project experience. Culture. Engineering judgment.

Develop technical reasoning and prepare evidence from your experience. No computer required.

Contents

How to use this reading edition 4

Chapter 1 - Go and data structure choices 8

Chapter 2 - Hashing, arrays, and sliding windows 16

Chapter 3 - Sorting, binary search, intervals, and heaps 26

Chapter 4 - Trees, graphs, and dependencies 37

Chapter 5 - Stacks, greedy choices, backtracking, and dynamic programming 49

Chapter 6 - LRU and weighted caches 59

Chapter 7 - Expiration, clocks, and concurrency 69

Chapter 8 - Rate limiting and rolling windows 81

Chapter 9 - Deduplication, counters, versions, and scheduling 90

Chapter 10 - Recommendations and streaming ranking 100

Chapter 11 - Modeling larger practical programs 112

Chapter 12 - Complete interview case studies 123

Chapter 13 Hiring manager interview preparation 136

Sources and scope 159

How to use this reading edition

This book prepares you to reason through a live Go coding interview with another senior engineer. The exercises are original study scenarios shaped by your preparation plan; they are not a verified list of Netflix questions.

Chapter 13 prepares the subsequent hiring-manager conversation for the L5 Developer Platform role. It contains researched question themes, answer ingredients, project-story guidance and questions for the manager. It can be read independently of the technical chapters.

The reading edition explains each problem directly. It gives a concrete input and expected output before introducing the technique, then traces state changes and shows Go code beside the reasoning. The separate conversation sessions remain available for teaching aloud, feedback, and one-question-at-a-time practice.

Read the problem before the implementation

A technique is an answer to a particular question. A map might store earlier indices for Two Sum, counts of prefix boundaries for exact-sum subarrays, or stable node pointers for LRU. Knowing only "use a map" does not tell you what to store or when to update it.

For each topic, identify the output, assumptions, and slow baseline first. Then read the example table as the state after each operation. Compare successive rows to see exactly which field changed and why. An invariant is a statement that must remain true; the explanation connects that statement to the implementation's branches.

Notation used throughout

Notation	Meaning
`[2, 7, 11, 15]`	Ordered array or sequence of values
`{A: 6, B: 7}`	Map from keys to values; display order is not semantic
`[(A, 4), (B, 7)]`	Ordered records or tuples for illustration
`nums[i:j]`	Elements i through j-1; end position is excluded
`[start, end)`	Interval including start but excluding end
`(now-W, now]`	Rolling interval excluding its left boundary
`[A, B]` in a cache trace	Most recent A, then least recent B

Illustrative tuples and maps are notation, not always literal Go syntax. Fenced Go blocks use actual Go declarations. Code omits package and import boilerplate for readability; examples using sorting, heap operations, or locks use `sort`, `container/heap`, or `sync`. The repository's example package assembles all blocks for compiler and behavior checks.

Presenting a solution in a live interview

Use these examples as a model for clear implementation: agree on the contract, name the state, explain the invariant, and then write the smallest solution that meets it. Handle the stated edge cases and say when an input assumption replaces validation. Brief code comments explain a boundary, mutation order, or ownership decision; the surrounding prose supplies the longer reasoning. Trace a normal case and a boundary case, and account for all retained state when explaining complexity. Add concurrency or a more elaborate index when the requirements call for it.

Reading code without running it

Before a loop, name what each variable means. For one row of the trace, follow the lookup, decision, and update in order. For prefix counting, for example, the map stores earlier boundaries before the current one is inserted. Reversing those lines changes the meaning and can introduce an empty-subarray bug.

Some algorithms are complete functions; small service examples explicitly identify their surrounding preconditions or omitted methods. The LRU, weighted cache, TTL map, transaction stack, and final metadata-cache baseline are assembled implementations. No paragraph asks you to discover an unstated solution before the book becomes useful.

Small examples keep the state visible. Diagrams show a specific relationship or transition, while tables give exact values. Shading highlights active state but is not required to understand the figure on a monochrome screen. Code blocks may continue across pages with a continuation label.

The route through the material

Chapters 1-5 develop representation, hashing, windows, ordering, traversal, stacks, and choice reasoning. Chapters 6-9 apply them to caches, deadlines, concurrency, limits, and event services. Chapter 10 covers ranking and rolling statistics. Chapter 11 models practical programs. Chapter 12 combines the concepts into two complete interview-style cases.

Chapter 13 shifts to project experience, engineering judgment, culture and working relationships. Its separate conversation session supports developing your own stories aloud without inventing facts or requiring an editor.

Read unfamiliar topics in order. On a later pass, you can cover the next table row and predict it, but the worked answers remain fully available. A change in assumptions is often the most useful check: negative numbers break a positive-only window proof; varying TTLs break insertion-order cleanup; falling scores break winner-only storage.

With this repository available to ChatGPT, ask to study or continue a topic. The tutor retrieves the separate conversation session and can use these reading examples too. No prompt copying, editor work, or code execution is required for independent reading or spoken learning. Progress is updated only from actual study, not from authoring changes to this book.

Chapter 1 - Go and data structure choices

Suppose a service receives viewing events and must answer questions about them. Should it store a slice, a map, or both? This chapter teaches you to choose by the question the program must answer, and to express that choice safely in Go. Later chapters build algorithms and caches from these same decisions.

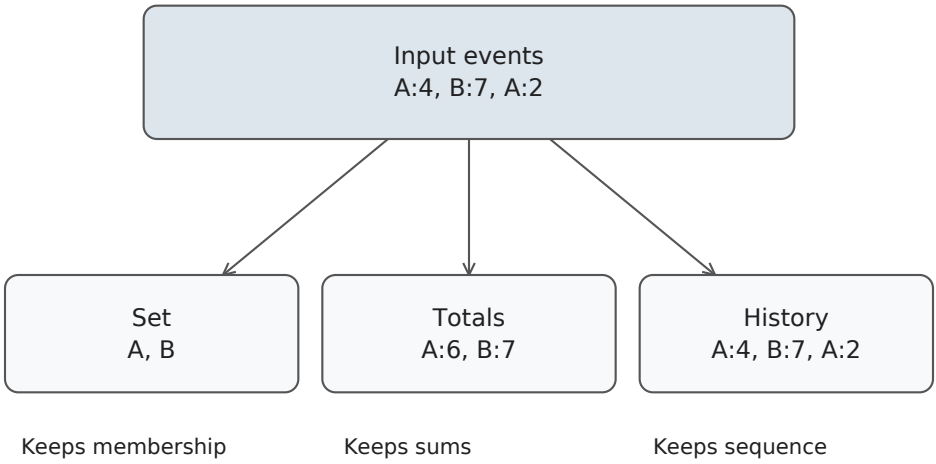
One input, three different problems

Use these ordered events, where each pair is `(title, minutes)`:

```
events = [("A", 4), ("B", 7), ("A", 2)]
```

The required output determines what information must survive. A membership query asks whether A appeared at all. A totals query asks how many minutes A received. A history query asks which events came last. They are different problems even though their input is identical.

ONE INPUT, THREE INFORMATION BUDGETS



Each arrow transforms the input; lost detail cannot be recovered.

The same events produce a membership set, a totals map, or an ordered history.

Required answer	State to retain	Example result
Has A appeared?	Set of title IDs	`{A, B}`; A is present
Total minutes per title	Map of sums	`{A: 6, B: 7}`
Last 2 events	Ordered records	`[(B, 7), (A, 2)]`

A set loses durations and order. A totals map loses individual contributions and order. Keeping the complete history allows all three queries, but uses more memory and may require repeated scans. An optimization is justified only if the discarded information cannot affect the required answer.

Go example: map lookup must distinguish zero from missing

A movie with 0 watched minutes is still a movie. Therefore a lookup returns both the record and a presence flag. Go's map lookup provides that flag directly.

```go
type Movie struct {
    Title   string
    Minutes int
}

func LookupMovie(movies map[string]Movie,
    id string) (Movie, bool) {
    movie, found := movies[id]
    return movie, found
}
```

For `movies = {"A": {Title: "A", Minutes: 0}}`, `LookupMovie(movies, "A")` returns the record and `true`; looking up B returns a zero-valued record and `false`. Testing `Minutes == 0` cannot distinguish them. The second result answers existence; the first answers value.

Use `make(map[string]Movie)` or `map[string]Movie{}` before assigning entries. Both create writable empty maps. `var movies map[string]Movie` creates a nil map: reading is allowed, but assigning into it panics. A set uses `map[string]struct{}` when only presence matters. Map iteration does not provide an agreed order; sort keys when output order is part of the contract.

Choosing a structure by its next operation

Assume n resident records and bounded-size keys. The following costs explain what each structure buys. They are not interchangeable guarantees.

Required operation	Starting structure	Cost and limitation
Access position i	Slice	O(1); middle deletion usually shifts O(n) elements
Find a key	Map	Expected O(1); no recency or sorted order
Take newest unfinished item	Stack	O(1) pop; useful for nested work
Take oldest pending item	Queue	O(1) head advance; needs arrival order
Repeatedly take minimum	Min heap	O(log n) remove; root is O(1)
Move known item to front	Doubly linked list	O(1) with node reference; lookup still needs a map
Latest version before time t	Sorted history	O(log n) search; arbitrary insertion may shift O(n)

Appending to a slice is amortized O(1): over n appends the total work is O(n), although an occasional append copies the existing array. Expected map cost also includes hashing the key; very long string keys are not free to process. Go does not supply a general ordered map in its standard library.

Worked example: an index makes deletion a joint operation

Suppose `items = ["A", "B", "C"]` and `position = {A: 0, B: 1, C: 2}`. The map makes lookup fast. Delete B while preserving display order: the slice becomes `["A", "C"]`, so C's index must become 1.

Step	Slice	Position map
Before deletion	`[A, B, C]`	`{A: 0, B: 1, C: 2}`
Shift C left	`[A, C]`	C's old index 2 is now invalid
Repair the index	`[A, C]`	`{A: 0, C: 1}`

The invariant is `items[position[id]] == id` for every resident ID. An invariant is simply a statement that must remain true after each public operation. Keeping two representations is useful only if updates preserve their agreement. LRU caches will apply this idea to a map and a linked list.

If order is irrelevant, replace the deleted item with the last item and shorten the slice. Only the moved item's index needs repair. That faster deletion changes behavior, so it must match the contract.

Go example: a returned slice can share your internal storage

Copying a slice copies its descriptor, not its elements. If `b := a`, then changing `b[0]` can change `a[0]`. A service that exposes its internal history this way lets callers corrupt its state.

```go
func CopyIDs(ids []string) []string {
    // Give the caller independent element storage.
    result := make([]string, len(ids))
    copy(result, ids)
    return result
}
```

For `ids = ["A", "B"]`, modifying `CopyIDs(ids)[0]` does not change ids. The copy costs O(n) time and space. This is a shallow copy:

if the elements themselves contain maps, slices, or pointers, the referenced objects can still be shared.

The same ownership issue appears when sorting. An in-place sort changes the caller's slice. Copy first if the API promises input preservation. Do not rely on an append happening to allocate a new backing array to provide isolation.

Bytes, runes, and the unit of a string problem

The question "longest substring" needs a definition of character. In Go, indexing a string returns a byte. Ranging over a string decodes runes, but the range index is a byte offset. Converting to `[]rune` gives positions measured in Unicode code points.

Text	UTF-8 bytes	Runes
`"A"`	`[65]`	1
`"é"`	`[195, 169]`	1
`"Aé"`	`[65, 195, 169]`	2

For ASCII-only anagrams, `[26]int` is a useful comparable map key. For arbitrary Unicode, that signature is insufficient. Even one rune is not always one human-perceived character: combining sequences can contain several code points. State the unit before selecting indexing rules.

Use `int64` when the permitted sums or timestamps need that width, and state that arithmetic must fit. Returning a boolean avoids inventing a sentinel such as -1 when -1 could be valid data. Invalid input should receive the error behavior promised by the API.

Putting the program together: unique IDs in first-arrival order

The registry accepts event IDs and returns whether an ID is new. It also returns a protected snapshot of distinct IDs in first-arrival order. The empty string is a valid ID, and duplicates do not move an ID. A set answers membership; a slice answers order.

```go
type Registry struct {
    seen  map[string]bool
    order []string
}

func (r *Registry) Accept(id string) bool {
    if r.seen[id] {
        return false
    }
    if r.seen == nil {
        // Initialize on first write; the zero value works.
        r.seen = make(map[string]bool)
    }
    r.seen[id] = true
    r.order = append(r.order, id)
    return true
}

func (r *Registry) History() []string {
    return CopyIDs(r.order)
}
```

Worked example: trace the registry's two views

Call	Return	Set	Ordered history
`Accept("A")`	true	`{A}`	`[A]`
`Accept("B")`	true	`{A, B}`	`[A, B]`
`Accept("A")`	false	`{A, B}`	`[A, B]`

The set and slice contain the same distinct IDs, and the slice stores each exactly once. Accept takes expected amortized O(1); History takes O(u) for u distinct IDs because it copies them. Adding removal would introduce new costs for preserving order. This is how a changed requirement can force a different representation.

Chapter 2 - Hashing, arrays, and sliding windows

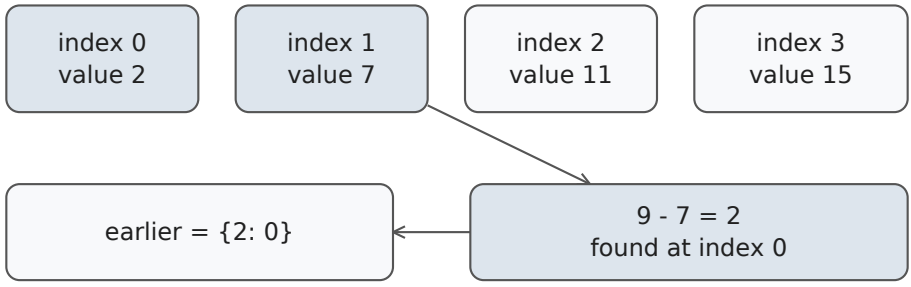
This chapter solves problems over ordered arrays: finding a pair, grouping equivalent words, finding a contiguous region, counting exact-sum subarrays, and combining products. A hash map is useful only after you decide what its keys and values mean. The examples make those meanings visible before showing code.

Two Sum: find two distinct positions

Problem. Given `nums = [2, 7, 11, 15]` and `target = 9`, return two different indices whose values add to 9. The answer is `[0, 1]`, because `2 + 7 = 9`. Return missing when no pair exists; do not reuse one element twice.

The slow baseline tries every pair, costing $O(n^2)$. When reading 7, the only question about the past is whether `9 - 7 = 2` occurred. A map from value to earlier index answers that question without another scan.

TWO SUM: CURRENT VALUE ASKS ABOUT THE PAST



At index 1, value 7 needs complement 2. The map points to the earlier index 0.

i	Value	Needed value	Map before lookup	Action
0	2	7	`{}`	Store `2: 0`
1	7	2	`{2: 0}`	Return `[0, 1]`

Go example: look up before inserting

```go
func PairSum(nums []int, target int) ([2]int, bool) {
    earlier := make(map[int]int)
    for i, value := range nums {
        // Search earlier positions before storing this one.
        if j, found := earlier[target-value]; found {
            return [2]int{j, i}, true
        }
        earlier[value] = i
    }
    return [2]int{}, false
}
```

Before iteration i, the map contains only earlier positions. Therefore a hit supplies a distinct index. For `[3, 3]`, target 6, the first 3 is stored and the second finds it. Inserting first would let a single `[3]` match itself. Time is expected O(n), space O(n), assuming arithmetic fits int.

Group Anagrams: define equivalence before hashing

Problem. Group `words = ["eat", "tea", "tan", "ate", "nat", "bat"]` by their letter counts. One valid result is `[["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]`. Group order is unspecified, while word order within a group follows input order.

Sorting each word's letters gives a stable signature. For lowercase ASCII a-z, a `[26]int` count array avoids sorting. "abb"

has counts a:1, b:2; "ab" has a:1, b:1. Presence alone would incorrectly treat them as equal.

```go
func AnagramGroups(words []string) [][]string {
    groups := make(map[[26]int][]string)
    for _, word := range words {
        // Counts preserve each letter's multiplicity.
        var key [26]int
        for i := 0; i < len(word); i++ {
            key[word[i]-'a']++
        }
        groups[key] = append(groups[key], word)
    }
    result := make([][]string, 0, len(groups))
    for _, group := range groups {
        result = append(result, group)
    }
    return result
}
```

The function assumes every byte is a-z. Arrays are comparable Go map keys; slices are not. Equal count arrays identify equal multisets of letters. Time is O(total input bytes), with fixed-size signature work per word. Arbitrary Unicode requires a different signature and a stated normalization policy.

Longest unique substring: keep a valid contiguous window

Problem. In `text = "ABBA"`, find the longest substring without a repeated rune. The answer is 2, from "AB" or "BA". A substring uses consecutive positions; sorting would change the problem.

Store the latest position of each rune and a left boundary. A repeated rune inside the window forces left past its earlier occurrence. An earlier occurrence outside the window does not force movement.

right	Rune	Previous position	left after repair	Window
0	A	Missing	0	`"A"`
1	B	Missing	0	`"AB"`
2	B	1	2	`"B"`
3	A	0	2	`"BA"`

GO

```
func UniqueLength(text string) int {
    runes := []rune(text)
    last := make(map[rune]int)
    left, best := 0, 0
    for right, r := range runes {
        // An old repeat must not move the window backward.
        if previous, found := last[r]; found &&
            previous >= left {
            left = previous + 1
        }
        last[r] = right
        if length := right - left + 1; length > best {
            best = length
        }
    }
    return best
}
```

The invariant is that `runes[left:right+1]` contains no duplicate. At the final A, assigning `left = 1` would move backward and reintroduce the two Bs. The guard prevents that. Both boundaries advance only, so time is O(n); rune conversion and the map use O(n) space.

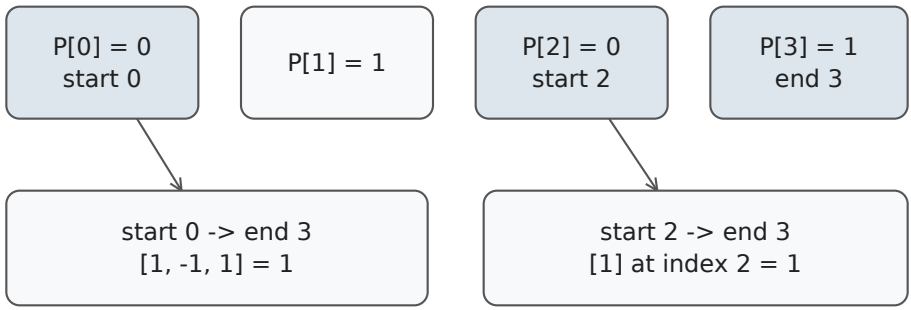
Worked example: why an existing prefix adds several answers

Problem. Count all nonempty contiguous subarrays summing to k, allowing negative values. For `nums = [1, -1, 1]`, `k = 1`, the answer is 3: `[1]` at index 0, `[1, -1, 1]`, and `[1]` at index 2. Distinct positions count separately even when values match.

A prefix sum is a cumulative total at a boundary. Define `P[0] = 0` and `P[j] = sum(nums[0:j])`. Then the sum of `nums[i:j]` is `P[j] - P[i]`. It equals k exactly when `P[i] = P[j] - k`.

For this input, `P = [0, 1, 0, 1]`. The two zero prefixes are at different boundaries. At the final prefix 1, subtracting either earlier zero produces a different subarray summing to 1.

PREFIX FREQUENCY COUNTS POSSIBLE STARTS



Two earlier zero prefixes -> add 2 at this step

At the final boundary, both earlier zero prefixes are valid starts. Each adds one distinct subarray.

The map stores **prefix value to number of earlier boundaries with that value**, not array value to count. Seed `{0: 1}` for the empty prefix before index 0.

i	Running prefix p	Need p-k	Earlier frequency	Total answers
0	1	0	1	1
1	0	-1	0	1
2	1	0	2	3

At index 1, recording the current prefix 0 changes its frequency from 1 to 2. At index 2, `frequency[0] == 2`, so add **2**, not 1. One start boundary gives the whole array; the other gives only its final element. A set would remember existence but lose this multiplicity.

Go example: count matching boundaries before recording this one

```go
func SubarrayCount(nums []int, k int) int {
    // The empty prefix lets a subarray start at index zero.
    frequency := map[int]int{0: 1}
    prefix, result := 0, 0
    for _, value := range nums {
        prefix += value
        // Count earlier boundaries before recording this one.
        result += frequency[prefix-k]
        frequency[prefix]++
    }
    return result
}
```

Before lookup, the map describes only earlier boundaries. Recording this boundary first would match a boundary with itself when `k == 0`, counting an empty subarray. For `[0, 0]`, k=0, the correct result is 3: each single zero and both zeros together. The second step legitimately adds 2 earlier starts.

This algorithm takes expected O(n) time and O(n) space. Prefix sums and the answer must fit int; use int64 if constraints require it. A positive-only sliding-window sum argument fails on signed values because removing a negative number increases the sum. Prefix subtraction remains valid regardless of signs.

Product Except Self: combine both sides of a position

Problem. For `nums = [2, 3, 4]`, return `[12, 8, 6]`, where each output excludes its own input element. Division is unavailable, and zeros must work.

Fill each output with the product strictly to its left. Then multiply by the product strictly to its right while walking backward. An empty side contributes 1, the multiplicative identity.

Position	Left product	Right product	Output
0	1	`3*4 = 12`	12
1	2	4	8
2	`2*3 = 6`	1	6

```go
func ProductsExceptSelf(nums []int64) []int64 {
    out := make([]int64, len(nums))
    left := int64(1)
    for i, value := range nums {
        // Write the product before including this element.
        out[i] = left
        left *= value
    }
    right := int64(1)
    for i := len(nums) - 1; i >= 0; i-- {
        // Combine products strictly on either side of i.
        out[i] *= right
        right *= nums[i]
    }
    return out
}
```

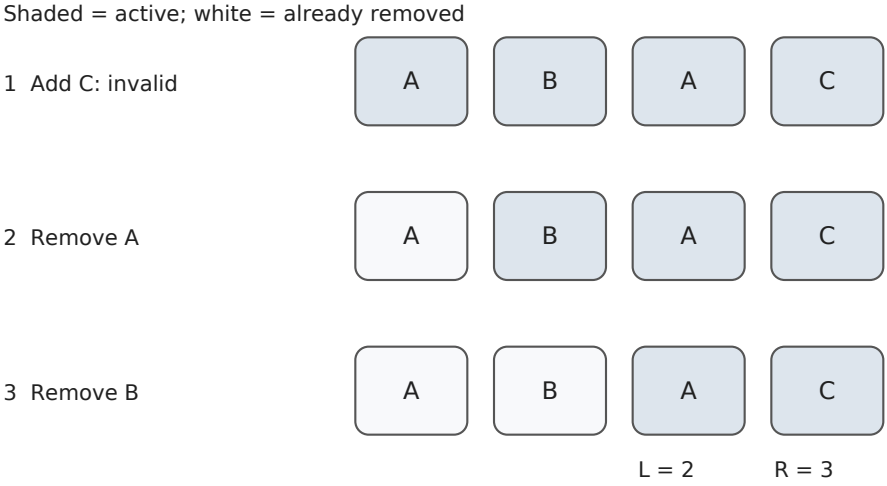
For `[2, 0, 4]`, the result is `[0, 8, 0]`; two zeros produce all zeros. Time is O(n), auxiliary space O(1) excluding output, assuming all intermediate products fit int64. The loop order matters: include the current value only after writing that position's strict-side product.

Worked example: at most two distinct titles

Problem. Find the longest contiguous region in `titles = ["A", "B", "A", "C", "C"]` containing at most 2 distinct IDs. The answer is 3. Repetitions are allowed, unlike the unique-substring problem.

A frequency map describes the current window. After adding C, counts are `{A: 2, B: 1, C: 1}`. Removing one A leaves 3 distinct titles; remove B too to repair the window to `[A, C]`.

WINDOW REPAIR: THE LEFT EDGE ONLY ADVANCES



Distinct counts: 3 -> 3 -> 2. Only the last row is valid.

The first C introduces a third distinct title. Removing A once and then B repairs the window.

Added	Window after repair	Counts	Best
A	`[A]`	`{A: 1}`	1
B	`[A, B]`	`{A: 1, B: 1}`	2
A	`[A, B, A]`	`{A: 2, B: 1}`	3
C	`[A, C]`	`{A: 1, C: 1}`	3
C	`[A, C, C]`	`{A: 1, C: 2}`	3

Putting the program together: repair before measuring

```go
func LongestTwo(titles []string) int {
    count := make(map[string]int)
    left, best := 0, 0
    for right, title := range titles {
        count[title]++
        for len(count) > 2 {
            old := titles[left]
            count[old]--
            if count[old] == 0 {
                // Only live keys count toward the limit.
                delete(count, old)
            }
            left++
        }
        if length := right - left + 1; length > best {
            best = length
        }
    }
    return best
}
```

Delete zero counts so `len(count)` means the number of distinct IDs. Use a loop because one removal may be insufficient. Measure only after validity is restored. Each element enters and leaves at most once, giving expected O(n) time. For the fixed bound of 2 titles, the map holds at most 3 keys during repair. Empty input returns 0.

The methods in this chapter retain different histories. Two Sum needs earlier positions; prefix counting needs earlier frequencies; a window needs only active-region state. Decide which future question the stored state must answer before choosing its representation.

Chapter 3 – Sorting, binary search, intervals, and heaps

This chapter solves problems where order lets you eliminate work. Sorting exposes directional comparisons, binary search finds a boundary, and a heap maintains only enough order to identify the next minimum. Begin by deciding which of these guarantees the output needs.

Three Sum: use sorted order to eliminate pairs

Problem. Return unique value triples summing to 0 in `nums = [-1, 0, 1, 2, -1, -4]`. The result is `[[-1, -1, 2], [-1, 0, 1]]`, in any triple order. Each triple uses distinct positions; duplicate triples by value appear only once.

The cubic baseline tries every triple. Sort to `[-4, -1, -1, 0, 1, 2]`, fix one value, and use two pointers on the suffix. If a sum is too small, increasing left is the only useful direction: decreasing right cannot raise it. If too large, decrease right. One scan per fixed value gives $O(n^2)$ time.

Fixed value	left value	right value	Sum	Action
-1	-1	2	0	Emit `[-1, -1, 2]`
-1	0	1	0	Emit `[-1, 0, 1]`

GO

```
func ThreeZero(nums []int) [][3]int {
    // Sorting a copy preserves the caller's input order.
    a := append([]int(nil), nums...)
    sort.Ints(a)
    var out [][3]int
    for i := 0; i+2 < len(a); i++ {
        if i > 0 && a[i] == a[i-1] {
            continue
        }
        left, right := i+1, len(a)-1
        for left < right {
            sum := a[i] + a[left] + a[right]
            if sum < 0 {
                left++
            } else if sum > 0 {
                right--
            } else {
                out = append(out,
                    [3]int{a[i], a[left], a[right]})
                // Skip values that would repeat this triple.
                x, y := a[left], a[right]
                for left < right && a[left] == x {
                    left++
                }
                for left < right && a[right] == y {
                    right--
                }
            }
        }
    }
    return out
}
```

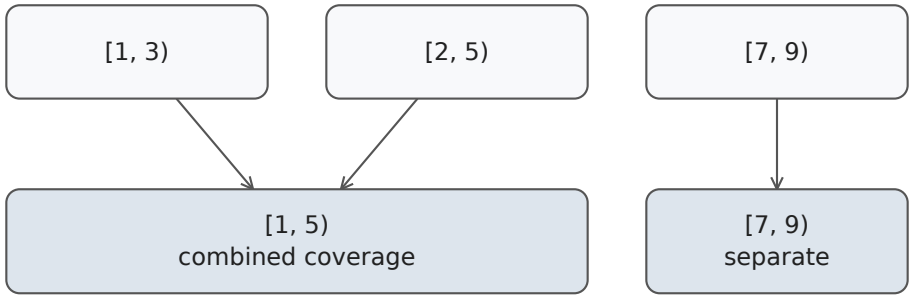
Skipping equal fixed values and equal emitted pair values prevents duplicates. The function copies input before sorting, costing O(n) auxiliary space plus output. Sorting destroys original position order, so an index-based output would need extra bookkeeping. Integer sums must fit int.

Merge intervals: retain the current coverage frontier

Problem. Combine overlapping coverage intervals `[[2, 5], [1, 3], [7, 9]]` into `[[1, 5], [7, 9]]`. For this exercise intervals are half-open `[start, end)`, and touching intervals also merge as continuous coverage. All intervals have `start < end`.

Sort by start. Keep the most recent merged interval. If the next start is at or before its end, extend the end; otherwise finalize it and start another. No later start can bridge a gap already passed.

MERGE SORTED COVERAGE, THEN KEEP THE GAP



Sorting puts overlapping intervals next to each other. The first two become one coverage interval.

```go
func MergeCoverage(input [][2]int) [][2]int {
    a := append([][2]int(nil), input...)
    sort.Slice(a, func(i, j int) bool {
        return a[i][0] < a[j][0]
    })
    var out [][2]int
    for _, interval := range a {
        last := len(out) - 1
        if last < 0 || interval[0] > out[last][1] {
            out = append(out, interval)
        } else if interval[1] > out[last][1] {
            // A nested interval must not shorten coverage.
            out[last][1] = interval[1]
        }
    }
    return out
}
```

Nested `[[1, 8], [2, 4]]` stays `[[1, 8]]`; do not shorten the end to 4. Empty input returns an empty result. Sorting dominates at O(n log n), and copies/output use O(n) space.

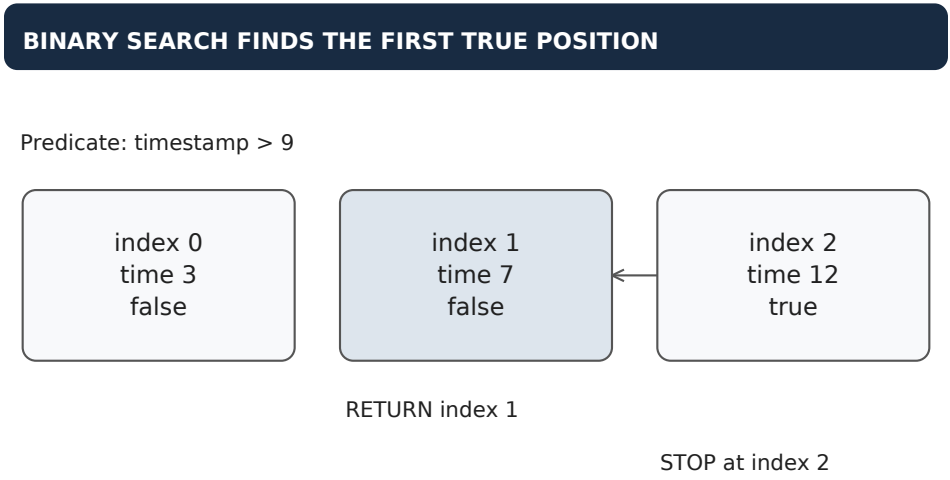
Related problem: maximum concurrent sessions. Union coverage and overlap counts need different state. For half-open `[1, 3)` and `[3, 5)`, simultaneous count is 1, even though this coverage contract merges them. A sweep represents a start as +1 and an end as -1, processing ends before starts at equal times.

Time	Changes	Active after changes
1	Start first	1
3	End first, start second	1
5	End second	0

Worked example: a predecessor is a boundary query

Problem. Metadata versions were written at `times = [3, 7, 12]`. At query time 9, return the latest version at or before 9: the one at 7. An exact-match search would miss it.

Search for the first timestamp greater than the query. For time 9, the predicate is `[false, false, true]`; its first true position is 2, so answer position 1. This false-then-true shape is the monotonicity binary search needs.



The first timestamp greater than 9 is 12. Its predecessor at 7 is the requested version.

Step	Unresolved range	mid	Time at mid	Change
1	`[0, 3)`	1	7	Eligible; `lo = 2`
2	`[2, 3)`	2	12	Too late; `hi = 2`
End	`[2, 2)`	-	-	Return position 1

Go example: search a half-open range

```go
type Version struct {
    At    int64
    Value string
}

func VersionAt(v []Version, at int64) (string, bool) {
    // Find the first version strictly after the query.
    lo, hi := 0, len(v)
    for lo < hi {
        mid := lo + (hi-lo)/2
        if v[mid].At <= at {
            lo = mid + 1
        } else {
            hi = mid
        }
    }
    if lo == 0 {
        return "", false
    }
    // Its predecessor is the latest eligible version.
    return v[lo-1].Value, true
}
```

Timestamps must be strictly increasing; equal-time writes are resolved before search. The unresolved range `[lo, hi)` shrinks every iteration. At termination lo is the first greater position,

possibly len(v). Query 2 returns missing, query 7 returns the version at 7, and query 20 returns the last version. Time is O(log m) for m versions, auxiliary space O(1).

Go's `sort.Search` expresses the same predicate and returns the length if no position is true. It does not return -1. Searching a rotated sorted array needs a different directional argument: identify the sorted half, then decide whether the target lies in it.

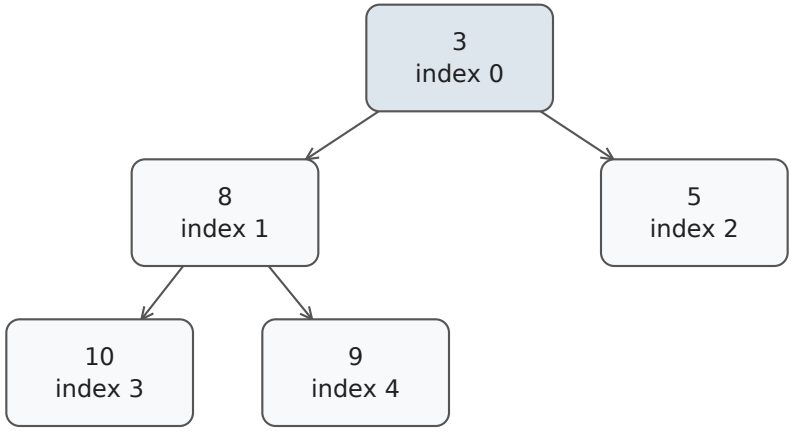
```go
func RotatedIndex(a []int, target int) int {
    lo, hi := 0, len(a)-1
    for lo <= hi {
        mid := lo + (hi-lo)/2
        if a[mid] == target {
            return mid
        }
        if a[lo] <= a[mid] {
            // The left half is sorted, so test its bounds.
            if a[lo] <= target && target < a[mid] {
                hi = mid - 1
            } else {
                lo = mid + 1
            }
        } else {
            if a[mid] < target && target <= a[hi] {
                lo = mid + 1
            } else {
                hi = mid - 1
            }
        }
    }
    return -1
}
```

For `[4, 5, 6, 7, 0, 1, 2]`, target 0, the first comparison discards the sorted left half `[4, 5, 6, 7]`. This code assumes distinct values and takes O(log n). Duplicates can hide which half is sorted and require a different algorithm with O(n) worst-case behavior.

What a heap guarantees

Problem. Repeatedly take the smallest pending priority without sorting the entire collection after every insertion. A min heap stores a complete binary tree in a slice. At index i, children are `2*i+1` and `2*i+2`; parent is `(i-1)/2` for i>0. Every parent is at most its children.

HEAP SLICE: ROOT MINIMUM, NOT FULL ORDER



The slice [3, 8, 5, 10, 9] is a valid min heap. Its root is smallest, but the entire slice is not sorted.

Insertion appends, then swaps upward while the parent is larger. Removal replaces the root with the last value, then swaps downward with the smaller child until order is repaired. Both take O(log n); inspecting the root takes O(1). Bottom-up heap construction is O(n).

Worked example: keep the three largest observations

For `stream = [4, 9, 2, 7, 5]`, maintain a min heap of at most k=3 winners. Its root is the weakest winner. Replace it only when the new value is larger.

Arrival	Winners shown sorted	Decision
4, 9, 2	`[2, 4, 9]`	Fill the slots
7	`[4, 7, 9]`	Replace root 2
5	`[5, 7, 9]`	Replace root 4

The table sorts for display; the heap need not. Here is the upward repair used by the integer heap:

```go
func PushMin(h []int, value int) []int {
    h = append(h, value)
    for i := len(h) - 1; i > 0; {
        parent := (i - 1) / 2
        if h[parent] <= h[i] {
            break
        }
        h[parent], h[i] = h[i], h[parent]
        i = parent
    }
    return h
}
```

Replacing the root requires downward repair:

```go
func RepairMinRoot(h []int) {
    for i := 0; ; {
        child := 2*i + 1
        if child >= len(h) {
            return
        }
        if child+1 < len(h) && h[child+1] < h[child] {
            // Repair through the smaller child.
            child++
        }
        if h[i] <= h[child] {
            return
        }
        h[i], h[child] = h[child], h[i]
        i = child
    }
}

func LargestK(values []int, k int) []int {
    if k <= 0 {
        return nil
    }
    var h []int
    for _, value := range values {
        if len(h) < k {
            h = PushMin(h, value)
        } else if value > h[0] {
            // The root is the weakest retained winner.
            h[0] = value
            RepairMinRoot(h)
        }
    }
    sort.Sort(sort.Reverse(sort.IntSlice(h)))
    return h
}
```

Duplicates count as separate observations: `[5, 5, 2]`, k=2, returns `[5, 5]`. Time is O(n log(k+1)) plus final O(k log k) sorting, with O(k) storage when k<=n. The kth largest is the smallest retained winner when n>=k. For top-k frequency, first count by ID, then compare distinct records by frequency and

the full tie rule. A worst-winner root must treat a larger ID as worse when smaller IDs win ties.

Putting the program together: time-based values

A time store maps each key to an ordered `[]Version`. Set accepts nondecreasing write times: a smaller time fails without mutation, equal time replaces the final value, and a larger time appends. Get applies VersionAt to that key's history. Empty values remain valid hits.

```go
type TimeStore map[string][]Version

func (s TimeStore) Set(key, value string, at int64) bool {
    history := s[key]
    if len(history) > 0 {
        last := len(history) - 1
        if at < history[last].At {
            return false
        }
        if at == history[last].At {
            history[last].Value = value
            return true
        }
    }
    s[key] = append(history, Version{At: at, Value: value})
    return true
}

func (s TimeStore) Get(key string, at int64) (string, bool) {
    return VersionAt(s[key], at)
}
```

Initialize with `make(TimeStore)`. After writes `(10, "HD")` and `(20, "UHD")` for quality, Get at 15 returns HD, at 20 returns UHD, and at 9 returns missing. Set is amortized O(1); Get is O(log m) for m versions of that key. Arbitrary write times require sorted insertion or a different ordered structure.

Chapter 4 - Trees, graphs, and dependencies

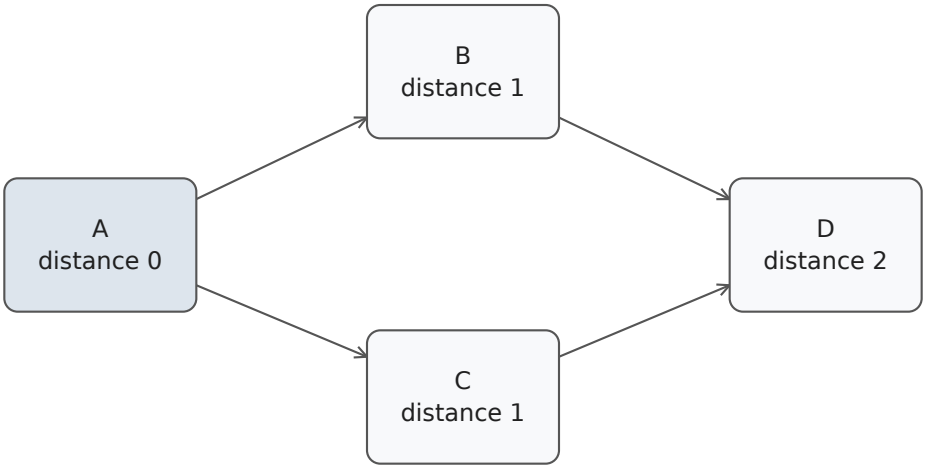
A graph models relationships rather than just a sequence. This chapter asks three different questions: how far away is a vertex, which vertices are connected, and when is a dependent job ready? All use a frontier of pending work, but the rule for adding work determines the algorithm.

Define vertices, edges, and the result

For recommendation links, a vertex is a title and an edge means a direct connection. For jobs, an edge from A to B means A must finish before B. For a grid, a vertex is a cell and edges join allowed neighboring cells. State whether edges are directed, weighted, or implicit.

The running graph is `edges = {A: [B, C], B: [D], C: [D], D: []}`. It has a shared descendant D. That is not a cycle: there is no path returning to its starting vertex.

BFS: DISCOVERY PROCEEDS ONE HOP AT A TIME



Every edge costs one hop. Mark D when it is enqueued.

The diamond graph offers two routes from A to D. Each edge costs one hop.

An adjacency list uses O(V+E) space and lets a traversal inspect each vertex and edge once. A matrix uses $O(V^2)$ space but offers O(1) edge-existence checks. A grid already supplies its adjacency rules, so storing a separate edge object for every neighbor is unnecessary.

Worked example: BFS gives shortest hop counts

Problem. Starting at A, return the minimum number of edges to each reachable vertex. The answer is `{A: 0, B: 1, C: 1, D: 2}`. All edges cost one hop.

A first-in-first-out queue processes distance 0, then distance 1, then distance 2. Mark a vertex when adding it, so two parents cannot enqueue it twice.

Removed	New discoveries	Pending queue
A	B:1, C:1	`[B, C]`
B	D:2	`[C, D]`
C	None; D is known	`[D]`
D	None	`[]`

Go example: record discovery on enqueue

```go
func HopDistances(edges map[string][]string,
    start string) map[string]int {
    distance := map[string]int{start: 0}
    queue := []string{start}
    for head := 0; head < len(queue); head++ {
        from := queue[head]
        for _, to := range edges[from] {
            if _, seen := distance[to]; seen {
                continue
            }
            // Mark now so other parents skip this vertex.
            distance[to] = distance[from] + 1
            queue = append(queue, to)
        }
    }
    return distance
}
```

The distance map also acts as the visited set. An unreachable vertex is absent; distance 0 belongs to the start. Missing adjacency entries mean no outgoing edges. Work is O(V+E) over reachable vertices, with O(V) additional space. Save a parent on first discovery if the requested output is the actual path, such as `[A, B, D]`.

BFS minimizes edge count, not arbitrary cost. A direct A-to-B edge of cost 10 loses to A-to-C-to-B with costs 1 and 1. For nonnegative weights, Dijkstra orders the frontier by tentative total cost in a min heap. It relaxes edges and skips stale heap distances. Negative weights invalidate its usual finalization argument.

Trees: BFS can group by level

Problem. Return levels of a binary tree with root 10, children 5 and 15, and a right child 12 beneath 5. The level output is `[[10], [5, 15], [12]]` regardless of whether the tree is a valid search tree.

Capture queue length at the beginning of each level. Nodes appended during that level belong to the next one.

```go
type TreeNode struct {
    Value       int
    Left, Right *TreeNode
}

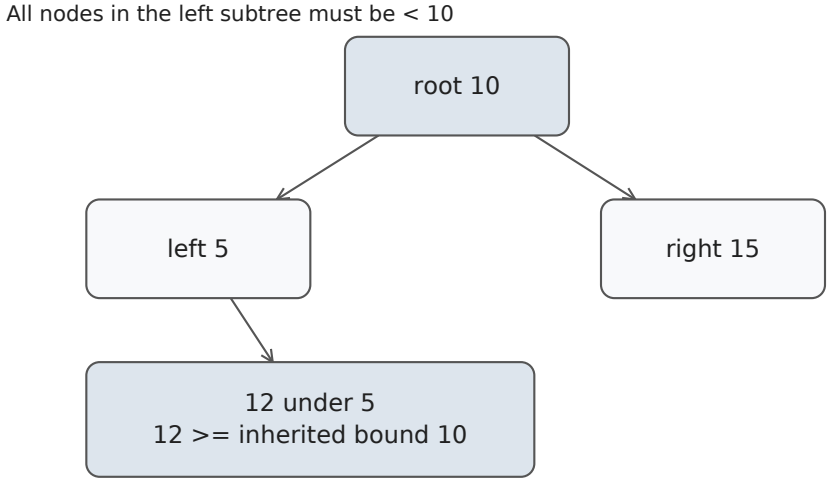
func TreeLevels(root *TreeNode) [][]int {
    if root == nil {
        return nil
    }
    queue := []*TreeNode{root}
    var levels [][]int
    for head := 0; head < len(queue); {
        // Newly enqueued children belong to the next level.
        end := len(queue)
        var level []int
        for head < end {
            n := queue[head]
            head++
            level = append(level, n.Value)
            if n.Left != nil {
                queue = append(queue, n.Left)
            }
            if n.Right != nil {
                queue = append(queue, n.Right)
            }
        }
        levels = append(levels, level)
    }
    return levels
}
```

Each node enters once: O(n) time. This simple head-index queue retains O(n) references until completion. A compacting queue can reduce retained storage toward maximum level width, but is unnecessary for a small traversal. The input must be a tree; cycles need visited state.

BST validation: local comparisons are insufficient

Problem. Is the same tree a binary search tree, with every left descendant smaller and every right descendant larger? The answer is false. The value 12 is greater than its parent 5, but belongs to the left subtree of 10 and must also be less than 10.

BST BOUNDS APPLY TO EVERY DESCENDANT



Node 12 satisfies its local parent comparison but violates the bound inherited from root 10.

Inorder traversal visits left, node, right. A valid BST with no duplicates produces strictly increasing values. Keep both the previous value and a flag saying whether it exists; the smallest integer is a valid value, not a safe sentinel.

```go
func IsBST(root *TreeNode) bool {
    previous, havePrevious := 0, false
    var visit func(*TreeNode) bool
    visit = func(n *TreeNode) bool {
        if n == nil {
            return true
        }
        if !visit(n.Left) {
            return false
        }
        // Inorder values must increase across subtree edges.
        if havePrevious && n.Value <= previous {
            return false
        }
        previous, havePrevious = n.Value, true
        return visit(n.Right)
    }
    return visit(root)
}
```

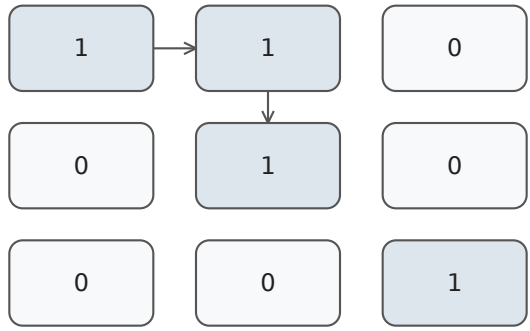
The counterexample visits `[5, 12, 10, 15]`; 10 fails after 12. Time is O(n), recursion space O(h) for tree height h, possibly n in a chain. Passing inherited lower and upper bounds is an equivalent approach.

Connected components: count islands with DFS

Problem. Count islands in `grid = ["110", "010", "001"]`, joining land only up, down, left, and right. The answer is 2. The final corner is diagonally adjacent but disconnected under this rule.

FOUR-WAY NEIGHBORS GIVE TWO ISLANDS

Grid = ["110", "010", "001"]



Bottom-right 1 is a separate island

Four-way neighbors form one three-cell island and one isolated corner. Diagonal contact does not connect them.

Scan every cell. Each unvisited land cell starts one new component; a stack explores and marks its entire component. This version mutates land `'1'` to water `'0'`, so callers needing preservation must copy each row first.

GO

```
func IslandCount(grid [][]byte) int {
    if len(grid) == 0 || len(grid[0]) == 0 {
        return 0
    }
    rows, cols := len(grid), len(grid[0])
    directions := [][2]int{{1, 0}, {-1, 0},
        {0, 1}, {0, -1}}
    count := 0
    for r := 0; r < rows; r++ {
        for c := 0; c < cols; c++ {
            if grid[r][c] != '1' {
                continue
            }
            count++
            grid[r][c] = '0'
            stack := [][2]int{{r, c}}
            for len(stack) > 0 {
                p := stack[len(stack)-1]
                stack = stack[:len(stack)-1]
                for _, d := range directions {
                    nr, nc := p[0]+d[0], p[1]+d[1]
                    if nr < 0 || nr >= rows ||
                        nc < 0 || nc >= cols ||
                        grid[nr][nc] != '1' {
                        continue
                    }
                    grid[nr][nc] = '0'
                    stack = append(stack, [2]int{nr, nc})
                }
            }
        }
    }
    return count
}
```

Assume a rectangular grid containing only '0' and '1'. Mark before pushing to avoid duplicate work. Each cell is visited once, so time is O(rows*cols); the stack can use O(rows*cols) space even though no separate visited array is allocated.

Worked example: a dependency queue means ready, not nearby

Problem. Execute jobs with prerequisites `A -> B`, `A -> C`, `B -> D`, `C -> D`. D must wait for both B and C. A possible result is `[A, B, C, D]`.

Indegree counts unfinished prerequisites: `{A: 0, B: 1, C: 1, D: 2}`. The ready queue initially contains A. Completing B alone leaves D blocked with indegree 1.

Completed	Updated prerequisite counts	Ready queue
A	B:0, C:0, D:2	`[B, C]`
B	D:1	`[C]`
C	D:0	`[D]`
D	All complete	`[]`

Putting the program together: topological ordering

This complete function uses integer job IDs `[0, n)`. Edges are `[prerequisite, dependent]`. Invalid IDs or cycles return false. Duplicate edges are counted and decremented consistently.

```go
func JobOrder(n int, edges [][2]int) ([]int, bool) {
    if n < 0 {
        return nil, false
    }
    next := make([][]int, n)
    pending := make([]int, n)
    for _, e := range edges {
        if e[0] < 0 || e[0] >= n ||
            e[1] < 0 || e[1] >= n {
            return nil, false
        }
        next[e[0]] = append(next[e[0]], e[1])
        pending[e[1]]++
    }
    var ready []int
    for id, count := range pending {
        if count == 0 {
            ready = append(ready, id)
        }
    }
    for head := 0; head < len(ready); head++ {
        for _, id := range next[ready[head]] {
            pending[id]--
            if pending[id] == 0 {
                // All prerequisites have now been processed.
                ready = append(ready, id)
            }
        }
    }
    return ready, len(ready) == n
}
```

Isolated jobs are included by initialization. Add `D -> A` and the cycle blocks complete output. An empty final queue alone does not prove success; compare output count with n. Time and space are O(V+E). A smallest-ready-ID requirement needs a heap instead of the queue.

Clone Graph: register identity before following cycles

Problem. Copy a graph without sharing its original nodes, preserving cycles and shared neighbors. Two distinct nodes may have the same label, so identity must use pointers rather than labels.

```go
type GraphNode struct {
    Label     string
    Neighbors []*GraphNode
}

func CloneGraph(start *GraphNode) *GraphNode {
    copied := make(map[*GraphNode]*GraphNode)
    var clone func(*GraphNode) *GraphNode
    clone = func(n *GraphNode) *GraphNode {
        if n == nil {
            return nil
        }
        if existing, found := copied[n]; found {
            return existing
        }
        duplicate := &GraphNode{Label: n.Label}
        // Register before recursion to terminate cycles.
        copied[n] = duplicate
        for _, neighbor := range n.Neighbors {
            duplicate.Neighbors = append(duplicate.Neighbors,
                clone(neighbor))
        }
        return duplicate
    }
    return clone(start)
}
```

For `A -> B -> A`, register A's copy before following B's back-edge so it reuses the copy. Late registration would recurse forever. Copying each reachable node once takes O(V+E) time and space. Keep discovery, readiness, and clone identity distinct.

Chapter 5 - Stacks, greedy choices, backtracking, and dynamic programming

Some problems ask you to match unfinished work; others ask you to choose a best combination. This chapter explains how to decide what state represents, whether a local choice is safe, and when different choice paths can reuse the same remaining answer.

Matching brackets: unfinished work has a last-in-first-out order

Problem. Validate nesting in `"([])"`. It is valid. `"([)]"` is invalid even though opening and closing counts match. A closing bracket must match the most recent unfinished opener.

Character in `"([])"`	Stack after action	Reason
`(`	`['(']`	Wait for closing round bracket
`[`	`['(', '[']`	The square opener is now innermost
`]`	`['(']`	Match and pop square opener
`)`	`[]`	Match and pop round opener

Stack cells contain the unmatched opening characters. The code below uses bytes and assumes the input contains only the six bracket characters. It rejects any other character.

Go example: match the top before popping

```go
func Balanced(text string) bool {
    pairs := map[byte]byte{')': '(', ']': '[', '}': '{'}
    var stack []byte
    for i := 0; i < len(text); i++ {
        ch := text[i]
        if ch == '(' || ch == '[' || ch == '{' {
            stack = append(stack, ch)
            continue
        }
        opener, valid := pairs[ch]
        if !valid || len(stack) == 0 ||
            stack[len(stack)-1] != opener {
            return false
        }
        stack = stack[:len(stack)-1]
    }
    return len(stack) == 0
}
```

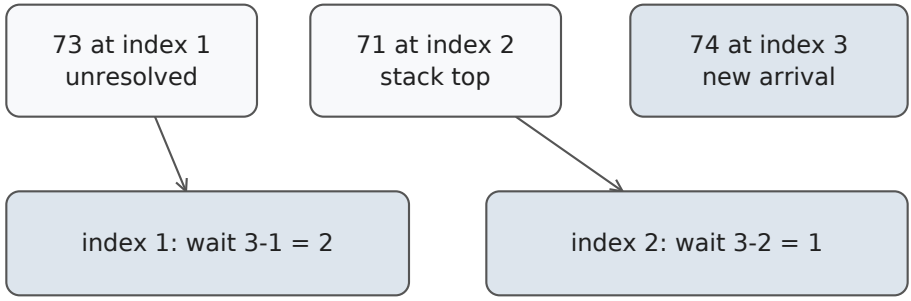
The stack is the exact sequence of unmatched openers. For `"([)]"`, ')' sees '[' at the top and immediately fails. An empty string is valid; a leading closer fails before indexing the stack. Each character is pushed or popped at most once: O(n) time and O(n) space.

Monotonic stacks: wait for a greater value

Problem. For `load = [70, 73, 71, 74]`, return how many later positions you must wait for a strictly greater value: `[1, 2, 1, 0]`. Zero means no later greater value.

Store unresolved indices, keeping their values nonincreasing from bottom to top. Arrival 74 resolves index 2 (71), then index 1 (73). Index 0 (70) was already resolved by 73.

ARRIVAL 74 RESOLVES TWO WAITING INDICES



Arrival 74 pops the unresolved loads 71 and 73. Their indices determine the waiting distances.

Arrival	Unresolved indices	Newly known waits
70 at 0	`[0]`	None
73 at 1	`[1]`	Index 0 waits 1
71 at 2	`[1, 2]`	None
74 at 3	`[3]`	Index 2 waits 1; index 1 waits 2

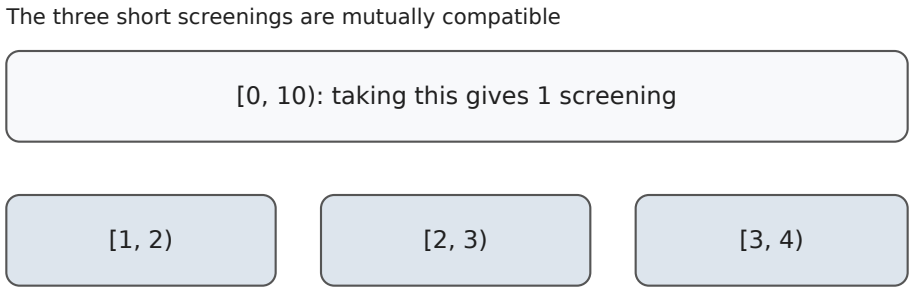
```go
func GreaterWait(values []int) []int {
    waits := make([]int, len(values))
    var stack []int
    for i, value := range values {
        for len(stack) > 0 {
            j := stack[len(stack)-1]
            if value <= values[j] {
                break
            }
            stack = stack[:len(stack)-1]
            // This is the first later value greater than j's.
            waits[j] = i - j
        }
        stack = append(stack, i)
    }
    return waits
}
```

Why is this the first greater value? If a greater value had arrived earlier, it would already have popped that unresolved index. Equal values do not resolve a strictly-greater query. Every index is pushed once and popped at most once, so the nested loop still totals O(n) work, with O(n) storage. Sliding-window maximum additionally needs expiration from the oldest end, so it uses a monotonic deque rather than this stack.

Greedy scheduling: the choice needs a proof

Problem. Select the largest number of nonoverlapping screenings from `[[0, 10], [1, 2], [2, 3], [3, 4]]`. Half-open intervals may touch. Choosing earliest start gives only `[0, 10]`; choosing earliest finish yields 3 short screenings.

EARLIEST FINISH LEAVES ROOM FOR MORE



The long early screening blocks three compatible short screenings. Earliest finish preserves room for later choices.

Sort by finish and take each compatible interval. In any optimal solution, replacing its first interval with the earliest-finishing one cannot obstruct later intervals, because it finishes no later. Repeat this exchange argument on the remaining schedule.

GO

```
func MostScreenings(input [][2]int) [][2]int {
    a := append([][2]int(nil), input...)
    sort.Slice(a, func(i, j int) bool {
        if a[i][1] != a[j][1] {
            return a[i][1] < a[j][1]
        }
        return a[i][0] < a[j][0]
    })
    var chosen [][2]int
    end, haveEnd := 0, false
    for _, interval := range a {
        // Earliest finish leaves room for later screenings.
        if !haveEnd || interval[0] >= end {
            chosen = append(chosen, interval)
            end, haveEnd = interval[1], true
        }
    }
    return chosen
}
```

Assume valid intervals with start<end. The boolean allows negative timestamps instead of using 0 as an accidental lower bound. Sorting costs O(n log n), storage O(n) for the preserved copy and output. If screenings carry different rewards and the objective becomes maximum total reward, this greedy proof no longer establishes optimality.

Backtracking: enumerate choices and undo each one

Problem. Choose exactly 2 titles within duration budget 5 from `duration = [2, 3, 4]`. Order does not matter. The only result is indices `[[0, 1]]`, totaling 5. The baseline is an explicit choice search, not a sorting trick.

A state contains the next available index, the chosen indices, and remaining budget. Use increasing indices to generate each combination once. After exploring a choice, remove it before exploring a sibling branch.

```go
func PlaylistChoices(duration []int,
    count, budget int) [][]int {
    if count < 0 || budget < 0 {
        return nil
    }
    var chosen []int
    var out [][]int
    var search func(int, int)
    search = func(start, remaining int) {
        if len(chosen) == count {
            // Later branches reuse chosen's backing array.
            snapshot := append([]int{}, chosen...)
            out = append(out, snapshot)
            return
        }
        needed := count - len(chosen)
        // Stop when too few titles remain to finish a choice.
        for i := start; i <= len(duration)-needed; i++ {
            if duration[i] > remaining {
                continue
            }
            chosen = append(chosen, i)
            search(i+1, remaining-duration[i])
            // Undo this choice before exploring its sibling.
            chosen = chosen[:len(chosen)-1]
        }
    }
    search(0, budget)
    return out
}
```

This assumes nonnegative durations; pruning an over-budget choice is unsafe if later negative contributions could repair it. The loop also stops when fewer than the needed number of titles remain. Copy each completed result because later mutations reuse chosen's backing array. Search can be exponential, and output can itself be exponential. Auxiliary recursion/path space is O(n), excluding saved answers.

A permutation problem would have a different contract: increasing indices would incorrectly discard different orders. A

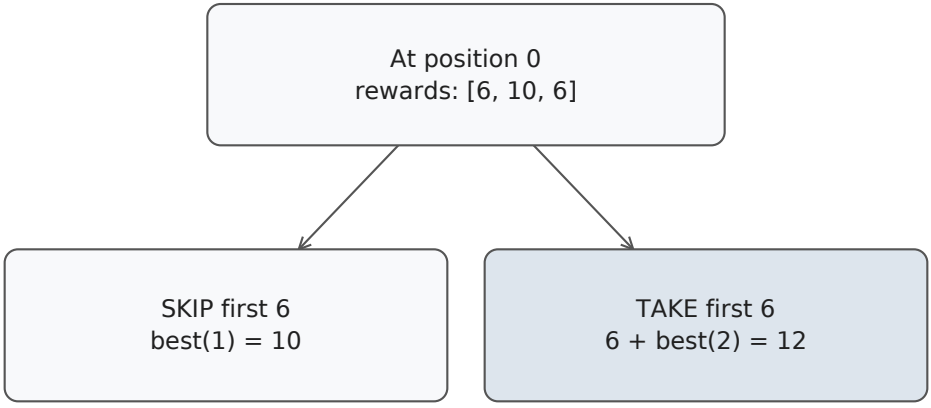
director restriction would need director state; index and budget alone would forget a constraint.

Worked example: derive a DP state from take or skip

Problem. Rewards `reward = [6, 10, 6]` occupy neighboring promotion slots. Select nonadjacent slots for maximum total; choosing nothing is allowed. The answer is 12 from indices `[0, 2]`. Greedily choosing the largest reward gives 10 and fails.

Define `best[i]` as the maximum reward using positions i onward. At i, either skip and obtain `best[i+1]`, or take and obtain `reward[i] + best[i+2]`. These are the complete legal choices. Beyond the array, best is 0.

COMPARE COMPLETE ALTERNATIVES, NOT LOCAL REWARDS



Choose 12: positions 0 and 2. Shading marks the winning branch.

Taking a slot jumps over its neighbor. Skipping advances one position; both lead to smaller suffix problems.

i, evaluated backward	Skip	Take	best[i]
2	0	`6+0 = 6`	6
1	6	`10+0 = 10`	10
0	10	`6+6 = 12`	12

A naive recursion reaches the same suffix through several decision paths. Memoization computes each suffix once; a backward loop does so directly. The state is sufficient because earlier choices impose no remaining condition after a legal entry to that suffix.

Putting the program together: two rolling suffix answers

```go
func BestNonAdjacent(reward []int64) int64 {
    var next, afterNext int64
    for i := len(reward) - 1; i >= 0; i-- {
        // next is best[i+1]; afterNext is best[i+2].
        take := reward[i] + afterNext
        current := next
        if take > current {
            current = take
        }
        afterNext = next
        next = current
    }
    return next
}
```

At iteration i, next means `best[i+1]` and afterNext means `best[i+2]`. Save the old next into afterNext before replacing next. Empty and all-negative inputs return 0 under the empty-selection contract. Time is O(n), space O(1); all sums must fit int64.

Worked example: value output and reconstructed output need different state

For `[4, 1, 1, 4]`, the full suffix table is `[8, 5, 4, 4, 0, 0]`. At index 0, taking gives `4 + best[2] = 8`, better than skipping's 5. Jump to index 2: skip its 1 because best[3]=4 is better than `1+best[4]=1`. Take index 3. Output indices are `[0, 3]`.

The rolling implementation returns 8 but deliberately discards the decisions needed to recover `[0, 3]`. Retain a table or decisions when the output requires them. Adding a selection limit or budget similarly expands the DP state. Count states times work per state; an O(nB) budget table depends on the numeric budget B and is pseudopolynomial.

Greedy discards choices using a proof; backtracking explores distinct choices; DP shares equivalent remaining problems. A tiny counterexample and an explicit state definition are more useful than guessing a technique from the story.

Chapter 6 - LRU and weighted caches

A cache avoids repeating expensive work, such as fetching movie metadata. Memory is limited, so it needs an eviction policy. This chapter implements least recently used eviction and then changes capacity from number of entries to total weight.

Define the observable cache behavior

Problem. Support `Put(key, value)` and `Get(key)` with capacity 2. A successful read or write makes its key most recent. When insertion exceeds capacity, remove the least recent resident. A missing read changes nothing. Zero is a valid value, and zero capacity stores nothing.

LRU means least recently used, not least frequently used. Many accesses yesterday do not outrank one access now. It favors temporal locality; a scan of unique keys can still displace useful residents.

Worked example: make recency visible

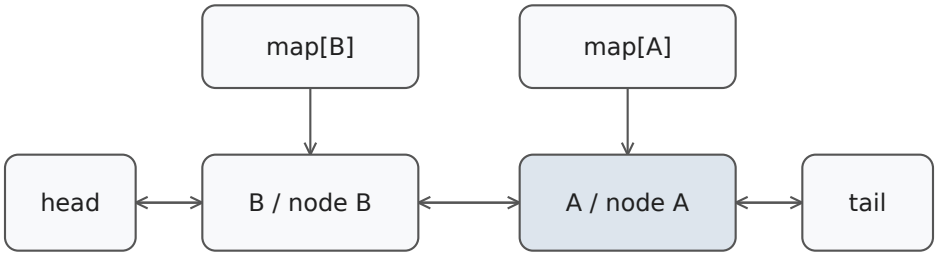
Order below runs from most recent to least recent:

Operation	Resident order	Result
`Put("A", 4)`	`[A]`	Insert A
`Put("B", 7)`	`[B, A]`	Cache full
`Get("A")`	`[A, B]`	Return `(4, true)`
`Put("C", 9)`	`[C, A]`	Evict B
`Put("A", 0)`	`[A, C]`	Update existing A

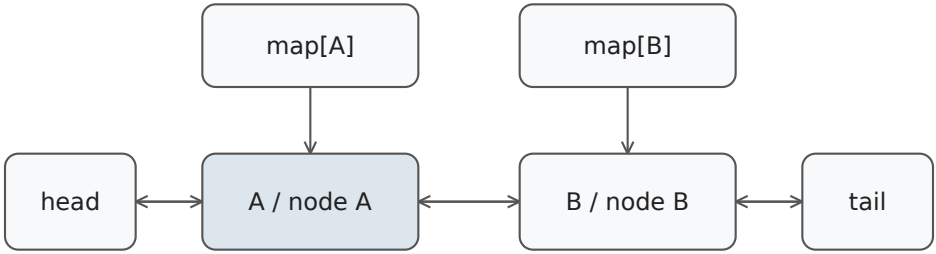
A map finds A quickly but supplies no meaningful recency order. A list supplies oldest and newest positions but cannot find an arbitrary key quickly. Use `map[key]*node` plus a doubly linked list. The map points to the same node even when that node moves.

LRU PROMOTION: SAME OBJECTS, DIFFERENT NEIGHBORS

BEFORE Get(A): B is most recent



AFTER Get(A): A is most recent



Double arrows = next/prev links. Sentinels are not in the map.

The map finds A's stable node. Promotion changes list order from [B, A] to [A, B].

Go example: define nodes and an initialized cache

Head and tail are empty sentinel nodes outside the map and capacity. Even an empty list has `head.next = tail` and `tail.prev = head`. That makes endpoint mutations use the same rules as middle mutations.

```go
type cacheNode struct {
    key        string
    value      int
    prev, next *cacheNode
}

type ReadCache struct {
    byKey      map[string]*cacheNode
    head, tail *cacheNode
    capacity   int
}

func NewReadCache(capacity int) *ReadCache {
    if capacity < 0 {
        capacity = 0
    }
    head, tail := &cacheNode{}, &cacheNode{}
    head.next, tail.prev = tail, head
    return &ReadCache{
        byKey: make(map[string]*cacheNode),
        head:  head, tail: tail, capacity: capacity,
    }
}
```

This constructor clamps negative capacity to zero. A stricter API could return an error instead. Use the constructor: a zero-valued ReadCache has no linked sentinels and is not initialized.

Three invariants connect the structures: every real node has one map entry; every map entry points to that real node; neighboring forward and backward links agree. Completed operations also satisfy resident count<=capacity.

Pointer helpers: detach, then insert

Detach joins a known node's neighbors. InsertFront places a detached node between head and the old first node. Because the map found the node and each node has both neighbors, neither operation searches the list.

```go
func detach(n *cacheNode) {
    // Join the neighbors before clearing this node's links.
    n.prev.next = n.next
    n.next.prev = n.prev
    n.prev, n.next = nil, nil
}

func insertFront(head, n *cacheNode) {
    first := head.next
    n.prev, n.next = head, first
    head.next = n
    first.prev = n
}

func (c *ReadCache) remove(n *cacheNode) {
    // Membership must change in both the list and the map.
    detach(n)
    delete(c.byKey, n.key)
}
```

For `head-B-A-tail`, detaching A connects B directly to tail. Inserting A at the front gives `head-A-B-tail`. The arrows in the figure stand for both directions; each assignment repairs one link. Never detach a sentinel or an already detached node.

Removal must update the map as well as the list. Leaving B's map entry after eviction would permit reads of an evicted value and later pointer corruption. Centralizing removal makes that joint mutation explicit.

Putting the program together: Get and Put

Get is lookup, promote, return. A miss performs no pointer edits.

```go
func (c *ReadCache) Get(key string) (int, bool) {
    n, found := c.byKey[key]
    if !found {
        return 0, false
    }
    detach(n)
    insertFront(c.head, n)
    return n.value, true
}
```

Put updates an existing node or adds one new node. Count-limited insertion can exceed capacity by only one, so one eviction is sufficient.

```go
func (c *ReadCache) Put(key string, value int) {
    if c.capacity == 0 {
        return
    }
    if n, found := c.byKey[key]; found {
        // An overwrite reuses the existing list node.
        n.value = value
        detach(n)
        insertFront(c.head, n)
        return
    }
    n := &cacheNode{key: key, value: value}
    c.byKey[key] = n
    insertFront(c.head, n)
    if len(c.byKey) > c.capacity {
        // One insertion can require only one count eviction.
        c.remove(c.tail.prev)
    }
}
```

Get A after the final table row returns `(0, true)`; Get B returns `(0, false)`. A repeated Put must not allocate a second A node. Both methods take expected O(1) time for bounded-size keys and values, with O(capacity) resident space. They are

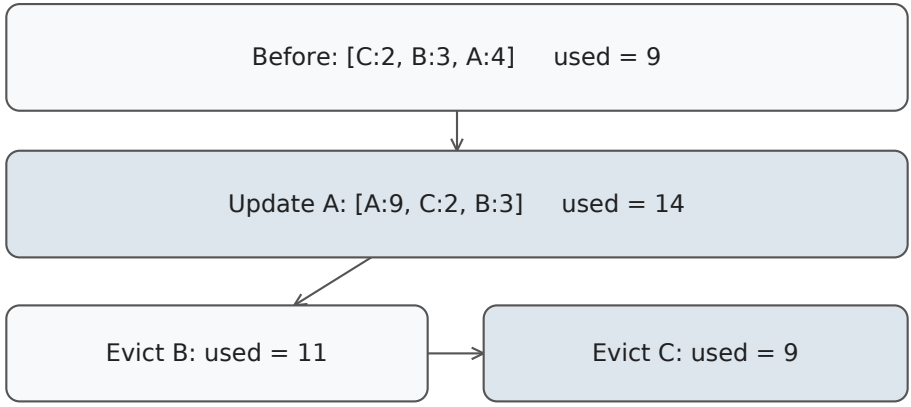
single-threaded; chapter 7 protects whole operations when concurrency is added.

Weighted capacity: number of entries is no longer the budget

Problem. Each entry has a positive weight, such as charged payload bytes. Total resident weight must stay at or below budget 10. Reject an item larger than the whole budget, leaving an old value and recency unchanged. Reject nonpositive weights in this variant.

Keep `used = sum(weights of residents)`. An overwrite subtracts its old charge before adding the new one. Eviction repeatedly removes the least recent entry until used<=budget. A byte-only policy must also account for actual node/map overhead if it promises a process-memory bound.

ONE WEIGHTED OVERWRITE CAN EVICT TWICE



Overwrite A from weight 4 to 9: total becomes 14. Evict B, then C, until only A remains at 9.

Worked example: one overwrite can require several victims

Initial order is `[C, B, A]`, weights `{C: 2, B: 3, A: 4}`, used=9. Overwrite A with weight 9: `used = 9-4+9 = 14`, order `[A, C, B]`. Removing B leaves 11, still over budget; removing C leaves 9.

Transition	Recency	Total weight
Initial	`[C, B, A]`	9
Update A to 9	`[A, C, B]`	14
Evict B	`[A, C]`	11
Evict C	`[A]`	9

If proposed weight were 11, reject before any mutation. The original `[C, B, A]` and used=9 survive. Input validation must precede destructive eviction.

Go example: weighted admission and shared removal

This extension reuses the node/list helpers, while maintaining its own weight accounting. It deliberately bypasses count-limited Put, because that method cannot update the weight map on its own.

```go
type WeightedCache struct {
    list         *ReadCache
    weights      map[string]int
    used, budget int
}

func NewWeightedCache(budget int) *WeightedCache {
    if budget < 0 {
        budget = 0
    }
    return &WeightedCache{
        list:    NewReadCache(0),
        weights: make(map[string]int), budget: budget,
    }
}

func (w *WeightedCache) Get(key string) (int, bool) {
    return w.list.Get(key)
}
```

The following Put returns whether admission succeeded. Weights and their intermediate total must fit int.

```go
func (w *WeightedCache) Put(key string,
    value, weight int) bool {
    // Rejection must preserve the old value and its recency.
    if weight <= 0 || weight > w.budget {
        return false
    }
    c := w.list
    n, found := c.byKey[key]
    if found {
        w.used -= w.weights[key]
        detach(n)
    } else {
        n = &cacheNode{key: key}
        c.byKey[key] = n
    }
    n.value = value
    insertFront(c.head, n)
    w.weights[key] = weight
    w.used += weight
    // A heavier overwrite may require several victims.
    for w.used > w.budget {
        victim := c.tail.prev
        w.used -= w.weights[victim.key]
        delete(w.weights, victim.key)
        c.remove(victim)
    }
    return true
}
```

The new node is most recent and individually fits the budget, so removing older residents eventually restores the invariant without needing to reject the admitted value. A Put evicting r residents costs O(1+r), not worst-case O(1). Get remains expected O(1). Positive weights bound the number of residents by the numeric budget; payload bytes alone still omit allocation overhead.

The central lesson is joint state: identity, recency, and total charge describe the same residents. Every branch that changes membership must update all affected views together.

Chapter 7 - Expiration, clocks, and concurrency

An LRU cache limits space but says nothing about freshness. Movie metadata can become stale while remaining popular. This chapter gives each write a lifetime, orders cleanup by deadlines, and protects multi-step mutations when several callers share the cache.

TTL: define exactly when a value becomes unavailable

Problem. Store `Put("A", "HD", ttl=5)` at time 0. A read at time 4 returns HD; a read at time 5 returns missing. TTL means time to live. Our deadline is `writeTime + ttl`, and a value is live only when `now < deadline`.

Overwriting resets the lifetime; reading does not extend it. Nonpositive TTL removes the key. The clock is injected, is non-nil, uses one agreed integer tick unit, and never goes backward. Deadline arithmetic and generation numbers must not overflow.

Time	Operation	Result
0	Put A with TTL 5	Deadline 5
4	Get A	`("HD", true)`
5	Get A	`("", false)`; remove A

This is expire-after-write. Expire-after-access is a different contract and would change deadlines on reads. Tests can set an injected clock to 4 or 5 directly, with no sleeping.

Go example: a map handles fresh visibility

Each current entry carries its value, absolute deadline, and write generation. A generation identifies one lifetime of one key, so delayed cleanup cannot confuse it with a replacement.

```go
type TimedValue struct {
    Value      string
    Deadline   int64
    Generation uint64
}

type Expiry struct {
    Key        string
    Deadline   int64
    Generation uint64
}

type TTLMap struct {
    current map[string]TimedValue
    due     ExpiryHeap
    next    uint64
    clock   func() int64
}

func NewTTLMap(clock func() int64) *TTLMap {
    return &TTLMap{
        current: make(map[string]TimedValue),
        clock:   clock,
    }
}
```

ExpiryHeap is the min heap defined below. Heap storage is separate from current entries; it may retain records for earlier lifetimes.

GO

```
func (c *TTLMap) Put(key, value string, ttl int64) {
    if ttl <= 0 {
        delete(c.current, key)
        return
    }
    deadline := c.clock() + ttl
    // A new generation distinguishes refreshes and reinserts.
    c.next++
    c.current[key] = TimedValue{
        Value: value, Deadline: deadline,
        Generation: c.next,
    }
    heap.Push(&c.due, Expiry{
        Key: key, Deadline: deadline, Generation: c.next,
    })
}

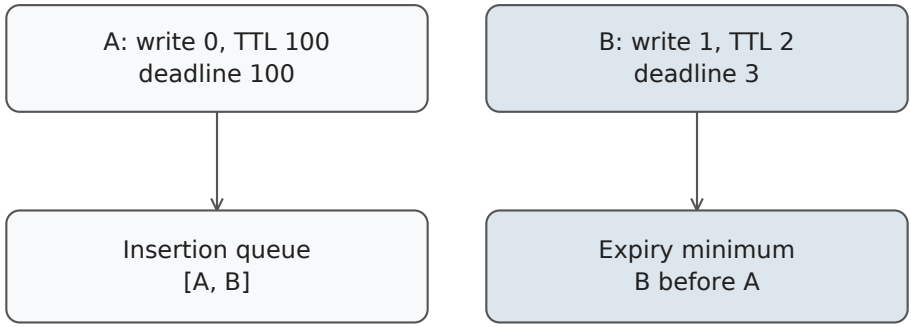
func (c *TTLMap) Get(key string) (string, bool) {
    entry, found := c.current[key]
    if !found {
        return "", false
    }
    if c.clock() >= entry.Deadline {
        delete(c.current, key)
        return "", false
    }
    return entry.Value, true
}
```

Get checks only its own key and takes expected O(1). Deleting on read is lazy expiration. It prevents stale reads but cannot reclaim an expired entry that nobody reads. Fresh visibility and prompt memory cleanup are separate requirements.

Cleanup needs deadline order, not insertion order

Write A at time 0 with TTL 100, then B at time 1 with TTL 2. Deadlines are `{A: 100, B: 3}`. A FIFO queue would see fresh A first and could leave expired B behind it. A deadline min heap exposes B first.

VARYING TTL BREAKS INSERTION ORDER



Insertion order [A, B] differs from deadline order [B, A]. Varying TTLs need a deadline index.

Cleanup choice	Work	Assumption or trade-off
Check on Get	Expected O(1)	Unread expired entries can remain
Periodic map scan	O(n) per scan	Simple; scan latency grows with residents
Deadline min heap	O(log h) per record removed	h includes stale records
FIFO deadline queue	Amortized O(1)	Deadlines must follow insertion order

The Go standard heap package supplies swaps and repairs through an interface. The receiver's Pop removes the final slice

element; `heap.Pop` first moves the minimum there. Calling the receiver's Pop directly would not mean remove-minimum.

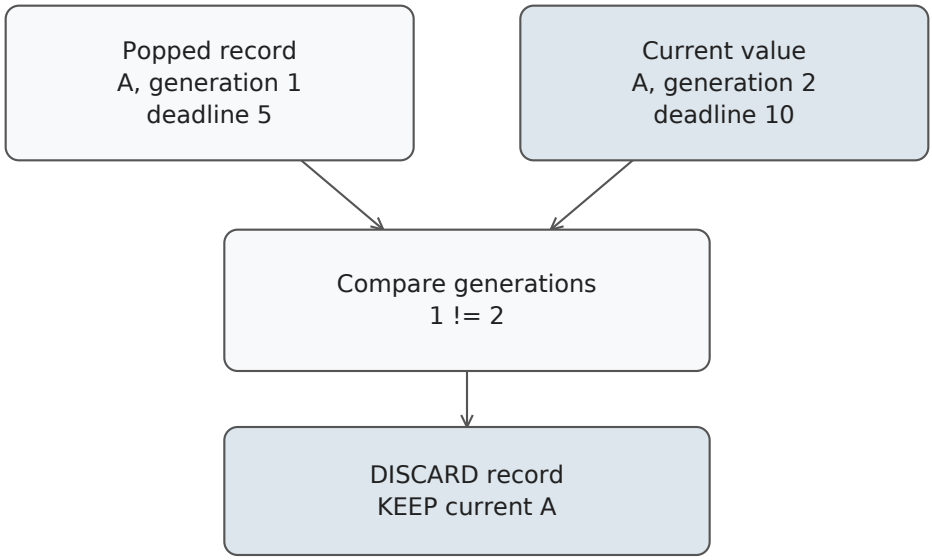
```go
type ExpiryHeap []Expiry

func (h ExpiryHeap) Len() int { return len(h) }
func (h ExpiryHeap) Less(i, j int) bool {
    return h[i].Deadline < h[j].Deadline
}
func (h ExpiryHeap) Swap(i, j int) {
    h[i], h[j] = h[j], h[i]
}
func (h *ExpiryHeap) Push(value any) {
    *h = append(*h, value.(Expiry))
}
func (h *ExpiryHeap) Pop() any {
    last := len(*h) - 1
    value := (*h)[last]
    // Release the key held in the unused backing-array slot.
    (*h)[last] = Expiry{}
    *h = (*h)[:last]
    return value
}
```

Worked example: an old deadline must not delete a new value

At time 0, write A with deadline 5 and generation 1. At time 3, refresh A with deadline 10 and generation 2. The map holds only the new value; the heap can hold both records.

STALE EXPIRY RECORDS CANNOT DELETE A REFRESH



Cleanup of generation 1 sees that current A is generation 2, so it ignores the old instruction.

Time	Current map entry	Heap action
0	`A: (old, 5, 1)`	Push `(A, 5, 1)`
3	`A: (new, 10, 2)`	Push `(A, 10, 2)`
5	`A: (new, 10, 2)`	Pop old record; ignore it
10	Missing	Remove current generation

Checking only the key at time 5 would delete fresh data. A global increasing generation prevents reuse after deletion and recreation. Comparing deadlines can identify this simple trace, but a generation is an explicit write identity even when two writes share a deadline.

```go
func ApplyExpiry(current map[string]TimedValue,
    record Expiry, now int64) bool {
    if record.Deadline > now {
        return false
    }
    entry, found := current[record.Key]
    // An old timer must not remove a newer write.
    if !found || entry.Generation != record.Generation {
        return false
    }
    delete(current, record.Key)
    return true
}
```

Putting the program together: drain only due records

```go
func (c *TTLMap) Cleanup() {
    now := c.clock()
    for len(c.due) > 0 && c.due[0].Deadline <= now {
        record := heap.Pop(&c.due).(Expiry)
        ApplyExpiry(c.current, record, now)
    }
}
```

Put takes O(log h); Get remains expected O(1); removing r due records costs O(r log(h+1)). Once the root is in the future, every remaining record is also in the future, so cleanup can stop.

Stale records consume memory. Rewriting one key a million times can produce a million heap records despite one resident map entry. An indexed heap can keep one deadline record per resident, or a periodic rebuild can recreate the heap from current entries. A rebuild costs O(n) and introduces a latency spike; include it in the design rather than claiming memory is always O(residents).

TTL plus LRU: freshness and recency are independent

Capacity 2 contains recency `[A, B]`, deadlines `{A: 5, B: 20}`. At time 5, Put C under an expired-first policy removes A before considering a live eviction. The result is `[C, B]`, even though A was most recent.

The map answers identity, the list answers least recent, and the heap answers earliest deadline. Removing an entry must update the map and list together. Chapter 12 gives a complete combined baseline. Cleanup can remove many records in one Put, so the combined Put has variable work.

Worked example: method safety does not make a sequence atomic

Two workers deduplicate event X using separate Get and Put calls. Each method can have a lock and still permit this sequence:

Step	Worker 1	Worker 2
1	Get X: missing	Waiting
2	Paused	Get X: missing
3	Process X	Process X
4	Put X	Put X

The check-and-record decision needs one atomic operation. Likewise LRU Get needs an exclusive lock because it moves list nodes, and TTL Get may delete expired entries. Protect the whole map/list/counter transition, not just the map access.

```go
type SafeReadCache struct {
    mu    sync.Mutex
    cache *ReadCache
}

func NewSafeReadCache(capacity int) *SafeReadCache {
    return &SafeReadCache{cache: NewReadCache(capacity)}
}

func (s *SafeReadCache) Get(key string) (int, bool) {
    // A cache hit changes recency, so Get needs a write lock.
    s.mu.Lock()
    defer s.mu.Unlock()
    return s.cache.Get(key)
}

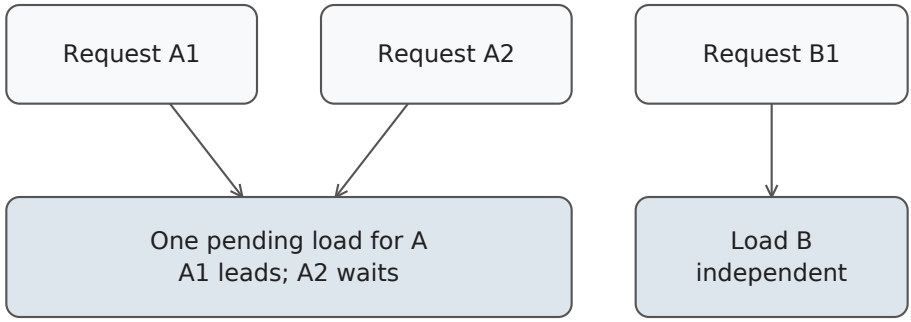
func (s *SafeReadCache) Put(key string, value int) {
    s.mu.Lock()
    defer s.mu.Unlock()
    s.cache.Put(key, value)
}
```

Initialize with NewSafeReadCache so the wrapper owns an initialized cache. All access to that cache goes through the wrapper. Helpers assume the caller already holds the lock; locking each helper recursively can deadlock. Do not copy a struct after its mutex has been used. Read/write locks are useful only when the protected read is truly read-only.

Single-flight: share one pending backend load per key

Problem. Three simultaneous misses for A should trigger one slow metadata load, while a load for B can proceed independently. Store a pending call per key. A leader loads outside the global lock; followers wait on its completion channel.

PENDING WORK IS SHARED BY KEY



The backend load runs outside the registry lock

Two requests for A share one pending load; an unrelated B load can run independently.

GO

```
type loadCall struct {
    done  chan struct{}
    value string
    err   error
}

type SharedLoader struct {
    mu     sync.Mutex
    active map[string]*loadCall
}

func (l *SharedLoader) Do(key string,
    load func() (string, error)) (string, error) {
    l.mu.Lock()
    if call, found := l.active[key]; found {
        l.mu.Unlock()
        // Wait without holding the lock needed by the leader.
        <-call.done
        return call.value, call.err
    }
    if l.active == nil {
        l.active = make(map[string]*loadCall)
    }
    call := &loadCall{done: make(chan struct{})}
    l.active[key] = call
    l.mu.Unlock()

    // Other keys can proceed while this backend call runs.
    value, err := load()
    l.mu.Lock()
    call.value, call.err = value, err
    delete(l.active, key)
    // Publish the result before waking every waiter.
    close(call.done)
    l.mu.Unlock()
    return value, err
}
```

This core assumes load returns normally, without panicking or recursively loading its own key. Followers receive both success and failure, and the pending record is cleared in either case. Channel completion publishes the result to waiters. This does not store a cache value or enforce TTL; compose it with fresh cache checks and admission.

Cancellation, panic policy, and invalidation during loading require additional rules. A version check can prevent a late obsolete result from repopulating an invalidated key. Sharding can reduce lock contention but typically changes a globally exact LRU policy into separate per-shard policies.

Chapter 8 - Rate limiting and rolling windows

A rate limiter decides whether a request may proceed. "100 per minute" can mean a fixed calendar bucket, an exact rolling minute, or a burst allowance replenished over time. These policies accept different sequences, so define the guarantee before choosing storage.

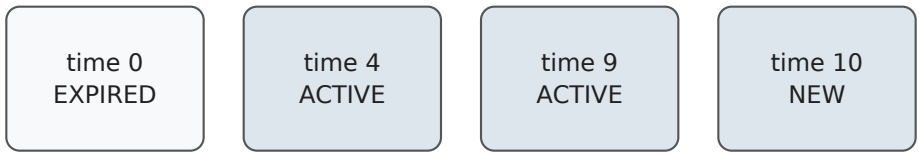
Exact rolling limit: what counts and what expires?

Problem. Allow at most L=3 accepted requests in the interval `(now-10, now]`. Rejected attempts do not count, and equal timestamps are allowed. Time is monotone admission time, not arbitrary client event time. Each user has independent state.

At now=10, a request accepted at 0 is outside the interval; one at 4 still counts. The left endpoint is open, the right endpoint closed. This convention settles the exact expiry comparison before implementation.

EXACT WINDOW: EXPIRE BEFORE TESTING CAPACITY

At now = 10, accepted timestamps must be in (0, 10].



1 Remove 0. 2 Count 4 and 9. 3 Accept the new 10.

A second request at 10 is rejected: three slots are already used.

For now=10 and width=10, timestamp 0 is excluded. Timestamps 4, 9, and the new 10 count.

Worked example: prune, check, then append

now	Active timestamps before decision	Decision	Active timestamps after
0	`[]`	Accept	`[0]`
4	`[0]`	Accept	`[0, 4]`
9	`[0, 4]`	Accept	`[0, 4, 9]`
10	`[4, 9]` after pruning 0	Accept	`[4, 9, 10]`
10	`[4, 9, 10]`	Reject	`[4, 9, 10]`

A chronological queue works because expired timestamps form a prefix. Remove that prefix, count survivors, then append only on acceptance. A rejected request must not extend lockout by entering the accepted log.

Go example: an exact log for one user

A head index avoids repeatedly shifting the entire queue. Periodic compaction copies surviving timestamps once enough consumed storage has accumulated.

```go
type WindowLimiter struct {
    Window   int64
    Limit    int
    accepted []int64
    head     int
}

func (l *WindowLimiter) Allow(now int64) bool {
    if l.Window <= 0 || l.Limit <= 0 {
        return false
    }
    cutoff := now - l.Window
    // The left window boundary is excluded.
    for l.head < len(l.accepted) &&
        l.accepted[l.head] <= cutoff {
        l.head++
    }
    if l.head > 0 && l.head*2 >= len(l.accepted) {
        // Reclaim consumed storage without copying each time.
        l.accepted = append([]int64(nil),
            l.accepted[l.head:]...)
        l.head = 0
    }
    if len(l.accepted)-l.head >= l.Limit {
        return false
    }
    // Rejected attempts never enter the accepted log.
    l.accepted = append(l.accepted, now)
    return true
}
```

Initialize `WindowLimiter{Window: 10, Limit: 3}`. Arithmetic must fit int64. Active storage is `accepted[head:]`; the consumed prefix does not count. Each accepted timestamp is added once and discarded once. Compaction is amortized over consumed

entries, so work is amortized O(1) per request; one call can still remove/copy O(L) entries. Retained storage is O(L).

Out-of-order times such as `[9, 1]` break the prefix argument: at now=10 and width=5, fresh 9 can hide expired 1. Use server admission time for this limiter. Late event-time analytics is a different problem.

Putting the program together: per-user state and atomic admission

Store a limiter per user. Protect creation, pruning, checking, and recording as one operation; two requests must not both claim the final slot. The first correct concurrent design uses one mutex.

```go
type UserLimiter struct {
    mu     sync.Mutex
    users  map[string]*WindowLimiter
    Window int64
    Limit  int
}

func (u *UserLimiter) Allow(user string, now int64) bool {
    // Creation and admission share one atomic decision.
    u.mu.Lock()
    defer u.mu.Unlock()
    if u.users == nil {
        u.users = make(map[string]*WindowLimiter)
    }
    limiter := u.users[user]
    if limiter == nil {
        limiter = &WindowLimiter{
            Window: u.Window, Limit: u.Limit,
        }
        u.users[user] = limiter
    }
    return limiter.Allow(now)
}
```

Time remains monotone for each user's serialized call order. With concurrent callers, timestamps sampled before acquiring the lock can arrive out of order; an injected server clock sampled under this lock avoids that problem. Immutable configuration and the lock preserve the admission invariant. U active users require O(U*L) timestamp storage, plus per-user maps and slice overhead. With 20 million full queues and L=100, bare 8-byte timestamps alone require 16 billion bytes. This is an illustrative bound, not a measured workload.

The sample retains users indefinitely. An idle-state policy can remove a user once every counted request has expired, because recreating an empty log preserves future behavior. Shards or per-user locks reduce contention, but creation and deletion of shared state must also be coordinated.

Fixed windows: compact counters permit boundary bursts

Problem. Count requests separately within fixed periods `[0, 10)`, `[10, 20)`, and so on, with L=3. Store only the bucket number and count. This rule intentionally differs from any rolling 10-second guarantee.

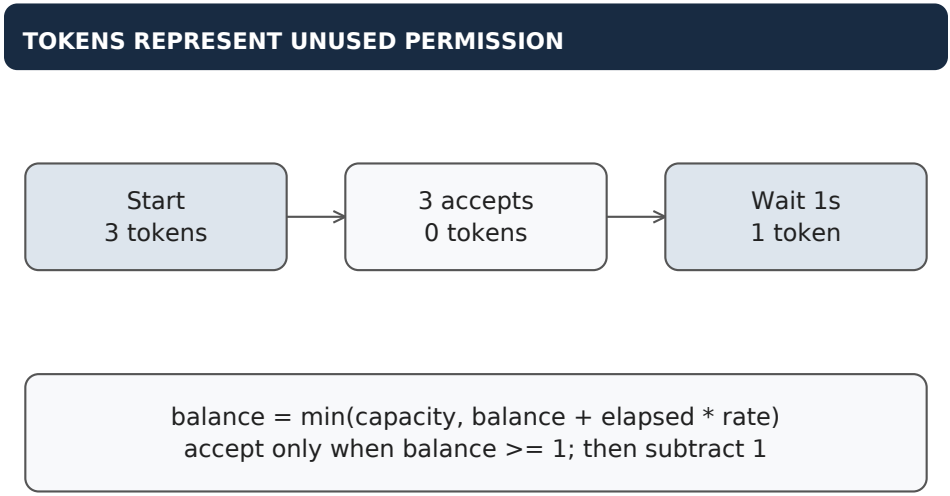
```go
type FixedLimiter struct {
    Window      int64
    Limit       int
    bucket      int64
    count       int
    initialized bool
}

func (f *FixedLimiter) Allow(now int64) bool {
    if f.Window <= 0 || f.Limit <= 0 || now < 0 {
        return false
    }
    bucket := now / f.Window
    if !f.initialized || bucket != f.bucket {
        // Each aligned window starts a separate allowance.
        f.bucket, f.count = bucket, 0
        f.initialized = true
    }
    if f.count >= f.Limit {
        return false
    }
    f.count++
    return true
}
```

This version assumes nonnegative monotone timestamps. Three requests at 9 and three at 10 are all accepted: 6 requests near one boundary, although each fixed bucket contains at most 3. Per request cost and storage are O(1). A two-bucket weighted estimate compresses further than an exact log, but cannot know precisely where requests occurred within a bucket.

Token bucket: burst capacity plus sustained refill

Problem. Permit an initial burst of C=3 requests, then replenish at r=1 token per second. Start full. Each accepted request costs one token. Waiting restores permission, capped at 3; it does not preserve exact request timestamps.



A bucket starts with three tokens. Requests spend them; elapsed time refills permission up to capacity.

Worked example: refill even when rejecting

now	Requests at that time	Balance after	Accepted
0	4	0	First 3
0.5	1	0.5	None
1	1	0	1
5	4	0	First 3

At time 1, refill only the 0.5 seconds since the last balance update. Reusing the whole interval since 0 would double-count refill after the rejected request at 0.5.

```go
type TokenBucket struct {
    Capacity, Rate float64
    tokens, last   float64
}

func NewTokenBucket(capacity, rate,
    now float64) *TokenBucket {
    return &TokenBucket{
        Capacity: capacity, Rate: rate,
        tokens: capacity, last: now,
    }
}

func (b *TokenBucket) Allow(now float64) bool {
    if now < b.last {
        now = b.last
    }
    b.tokens += (now - b.last) * b.Rate
    if b.tokens > b.Capacity {
        b.tokens = b.Capacity
    }
    // Advance on rejection too; never count refill twice.
    b.last = now
    if b.tokens < 1 {
        return false
    }
    b.tokens--
    return true
}
```

Assume finite capacity>=1 and rate>=0. This single-threaded core clamps a backward clock defensively. Floating-point rounding can affect near-boundary decisions; exact fixed units with a retained remainder are an alternative. O(1) state and time are sufficient because the contract is a balance, not an exact recent-event count.

Policy	State	Guarantee
Exact rolling log	Accepted timestamps	At most L in every width-W window
Fixed counter	Bucket and count	At most L in each aligned bucket
Token bucket	Balance and last refill	Burst up to C, then refill rate r

With C=3 and r=1/s, a bucket can admit 3 immediately and more as time passes; it does not mean "at most 3 in every rolling 10 seconds." Over elapsed interval T, available permission is bounded by starting balance plus r*T, with capacity limiting stored permission.

For several servers, independent per-server state can multiply a user's allowance. A shared atomic decision, stable user ownership, or explicitly partitioned quotas changes that. Define behavior when shared state is unavailable. A distributed limit is an additional contract, not something a local mutex establishes.

Chapter 9 - Deduplication, counters, versions, and scheduling

Event services combine identity with time. This chapter suppresses repeated deliveries, counts recent hits, queries historical values, and releases scheduled work. The same data structures recur, but each operation has a distinct promise about what must happen once, expire, or remain queryable.

Deduplication: an ID identifies one logical event

Problem. Deliveries `[(e1, A), (e2, B), (e1, A)]` should produce new-event results `[true, true, false]`. Different IDs with identical payloads remain different events. Repeated IDs are assumed to carry identical payloads.

A set works for lifetime-long suppression. For a 10-minute horizon, store each ID's expiry from its first accepted processing time. A duplicate does not refresh that deadline in this contract. An ID arriving after expiry can be accepted again.

Processing time	Delivery	Stored deadline	New?
0	e1	600	Yes
599	e1	600, unchanged	No
600	e1	1200	Yes

Go example: check and record as one decision

```go
type Deduper struct {
    mu      sync.Mutex
    expiry  map[string]int64
    Horizon int64
}

func (d *Deduper) Accept(id string, now int64) bool {
    d.mu.Lock()
    defer d.mu.Unlock()
    if d.Horizon <= 0 {
        return true
    }
    if deadline, found := d.expiry[id]; found &&
        now < deadline {
        // A duplicate does not extend its suppression window.
        return false
    }
    if d.expiry == nil {
        d.expiry = make(map[string]int64)
    }
    d.expiry[id] = now + d.Horizon
    return true
}
```

Assume monotone processing time and fitting deadline arithmetic. Accept is expected O(1). Expired IDs remain until reused in this simple implementation; scan or deadline-heap cleanup is required for bounded retention. Multiple servers need shared identity coordination, not independent local sets.

This is duplicate suppression, not durable exactly-once effects. Mark before a callback and a crash can suppress unfinished work. Mark after a callback and a crash can permit the effect twice. An idempotent destination or a transaction connecting effect and marker must address that gap.

Hit counters: aggregate equal-time arrivals

Problem. Record hits at `times = [100, 101, 101, 150]`. At now=150, the rolling interval `(90, 150]` contains 4 hits. At now=160, `(100, 160]` contains 3. The query's now value is part of the input.

With integer-second timestamps, repeated hits in one second share a bucket. A running total avoids recounting every retained event. An expiry queue preserves the contribution to subtract.

Bucket	Count at now=150	Included at now=160?
100	1	No; exact left boundary
101	2	Yes
150	1	Yes

GO

```
type HitBucket struct {
    At    int64
    Count int
}

type HitCounter struct {
    Window  int64
    buckets []HitBucket
    total   int
}

func (c *HitCounter) prune(now int64) {
    expired := 0
    for expired < len(c.buckets) &&
        c.buckets[expired].At <= now-c.Window {
        // Reverse each expired bucket's contribution once.
        c.total -= c.buckets[expired].Count
        expired++
    }
    c.buckets = c.buckets[expired:]
}

func (c *HitCounter) Record(now int64) {
    c.prune(now)
    last := len(c.buckets) - 1
    if last >= 0 && c.buckets[last].At == now {
        c.buckets[last].Count++
    } else {
        c.buckets = append(c.buckets, HitBucket{now, 1})
    }
    c.total++
}

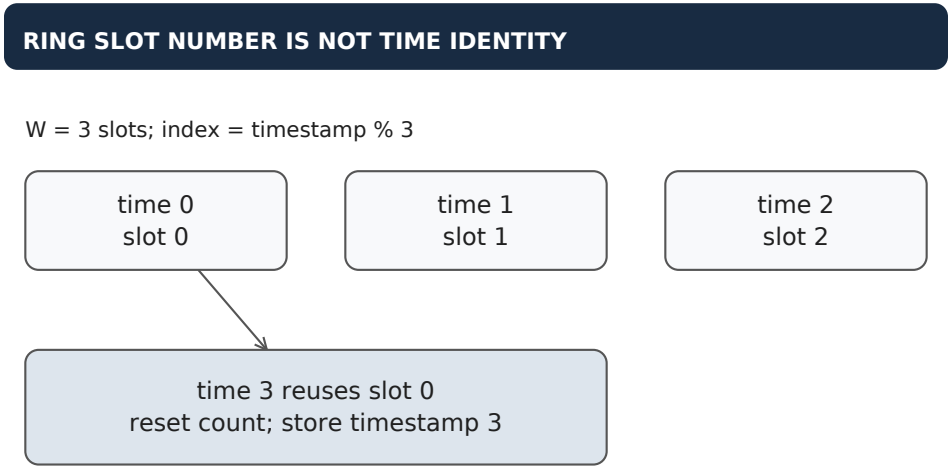
func (c *HitCounter) Count(now int64) int {
    c.prune(now)
    return c.total
}
```

Assume Window>0, nondecreasing times across Record and Count, and integer counts that fit. Each bucket enters/leaves once, giving amortized O(1) work plus allocation costs. The slice version can retain backing storage and needs compaction

in a long-lived service. For integer-second fixed width W, at most W active buckets exist.

Worked example: a ring slot needs an absolute timestamp

A ring of W=3 slots can use `slot = time % 3`. Time 0 and time 3 both use slot 0. Their equal slot number does not make them the same bucket. Store `(absolute time, count)` in every slot and reset count when the timestamp changes.



The same physical slot is reused by times 0 and 3. Its absolute timestamp distinguishes the current cycle.

For arbitrary subsecond queries, whole-second buckets lose precision at a partial boundary. For variable query horizons, one fixed running total is insufficient; retain enough history for the largest supported horizon and use scans or prefix summaries. Out-of-order event times also invalidate FIFO pruning unless handled explicitly.

Time-based values: retain history for predecessor queries

Problem. After `quality = [(10, "HD"), (20, "UHD")]`, a query at 15 returns HD, at 25 returns UHD, and at 9 returns missing. A map of only the latest value cannot answer the historical query.

Chapter 3's TimeStore keeps an ordered Version slice per key. Get finds the first timestamp greater than the query and returns its predecessor. Equal-time writes replace the latest version; earlier writes are rejected under that append-only contract.

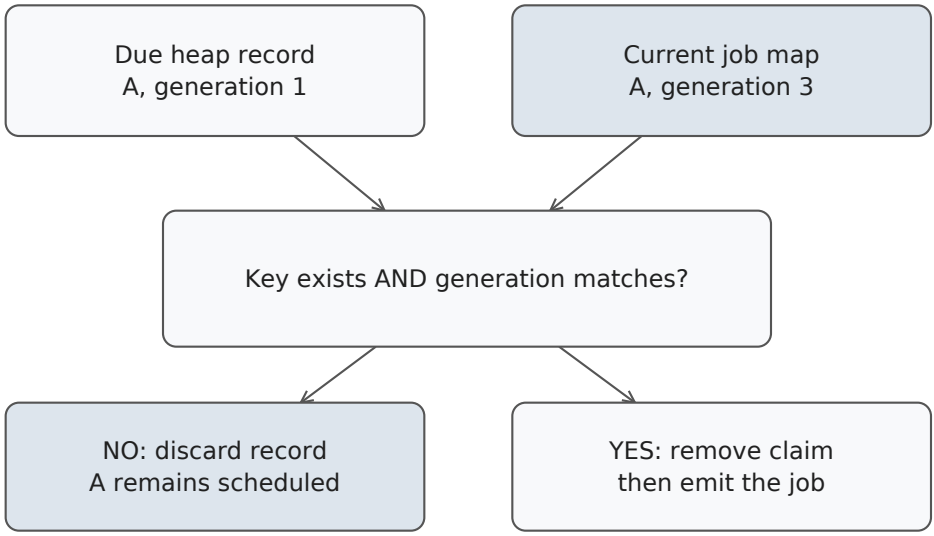
A deletion at time 30 requires an explicit tombstone version if old queries must remain possible. Physically erasing all history would make a time-15 query incorrect. Retention is a separate rule: expiring history may intentionally make old answers unavailable.

Scheduling: earliest due record plus current task identity

Problem. Schedule A for time 5, cancel it, then schedule a new A for time 9. At time 5 the old record must not run the new task. At time 9 the new generation can be claimed once.

A deadline min heap answers which record is due next. A current-generation map answers whether that record is still authorized. Cancellation deletes the map entry; rescheduling assigns a fresh global generation. Heap records can remain as stale work.

DRAINING A HEAP IS NOT THE SAME AS RUNNING A JOB



For this record, follow NO. Shading marks the actual outcome.

The due A generation 1 is stale after rescheduling. Only current generation 3 may claim A.

Worked example: claim before returning the task

Action	Current map	Heap records
Schedule A at 5	`{A: 1}`	`[(5, A, 1)]`
Cancel A	`{}`	Old record remains
Schedule A at 9	`{A: 3}`	Old plus `(9, A, 3)`
RunReady at 5	`{A: 3}`	Discard generation 1
RunReady at 9	`{}`	Claim generation 3

The skipped generation 2 is harmless; only uniqueness matters. Claiming removes current identity before exposing the

job, preventing another worker from claiming it again in this in-memory lifecycle.

Go example: validate due time and generation

```go
type JobRecord struct {
    ID         string
    Due        int64
    Generation uint64
}

func ClaimJob(current map[string]uint64,
    record JobRecord, now int64) bool {
    if record.Due > now {
        return false
    }
    generation, found := current[record.ID]
    if !found || generation != record.Generation {
        return false
    }
    // Remove authorization before handing work to a caller.
    delete(current, record.ID)
    return true
}
```

This helper is the authorization step, not a full scheduler. The surrounding scheduler must hold its lock across heap removal and ClaimJob, allocate unique generations, and execute callbacks after releasing the lock. A repeated claim fails because the current entry was removed.

Putting the program together: a cancellable scheduler

Schedule assigns a generation, records it in current, and pushes `(due, ID, generation)` into a min heap. Cancel deletes current[ID]. RunReady repeatedly removes due records and returns only successful claims. A future root stops the loop. The heap interface has the same shape as ExpiryHeap, with Due as the comparison field.

Here is the complete due-draining step using the same expiry-record representation as chapter 7. The current map still describes tasks, not cache entries.

```go
func ReadyJobs(due *ExpiryHeap,
    current map[string]uint64, now int64) []string {
    var ready []string
    for due.Len() > 0 && (*due)[0].Deadline <= now {
        record := heap.Pop(due).(Expiry)
        job := JobRecord{
            ID: record.Key, Due: record.Deadline,
            Generation: record.Generation,
        }
        // Cancelled or replaced generations are discarded.
        if ClaimJob(current, job, now) {
            ready = append(ready, job.ID)
        }
    }
    return ready
}
```

Removing r records costs O(r log(h+1)); some records may be stale, so count them too. Cancellation is expected O(1) with lazy invalidation, but stale heap memory can grow. Equal-deadline order is unspecified here; add an ID or sequence tie rule if required.

A claimed task is not necessarily a completed task. A worker crash after claim loses it in this simple design. Durable retries need lifecycle states, leases or acknowledgements, and idempotent effects. Those guarantees cannot be obtained by renaming a local heap "exactly once."

Chapter 10 - Recommendations and streaming ranking

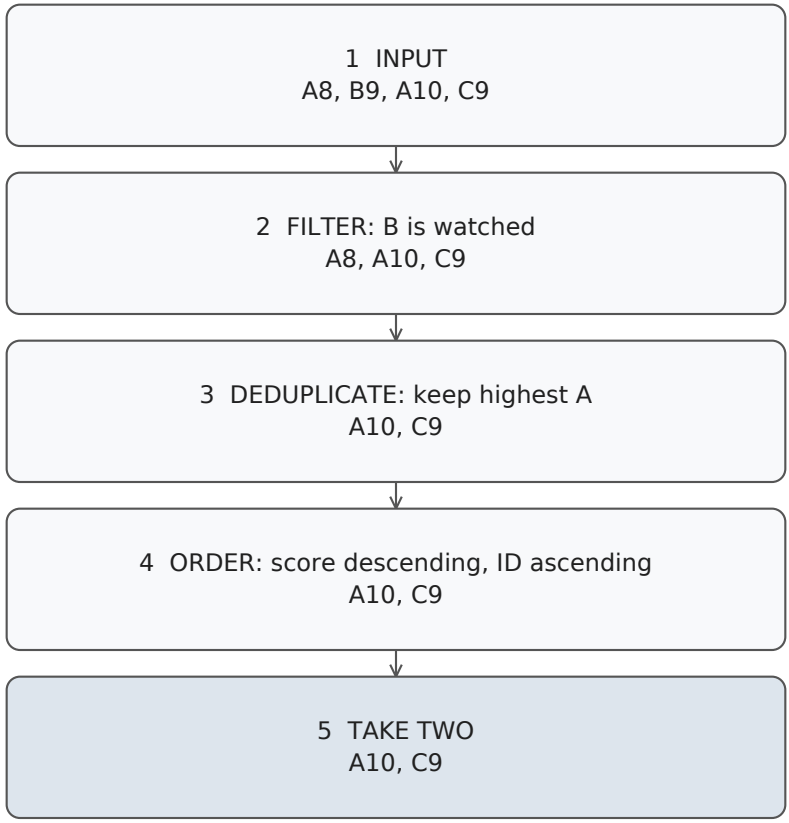
A recommendation exercise can be a precise coding problem without implementing machine learning. This chapter ranks supplied candidates, applies simple diversity rules, derives a toy collaborative score, and maintains rolling watch statistics. Each has a different input contract and state requirement.

Separate eligibility, identity, score, and order

Problem. Candidates have ID, score, genre, and eligibility. Exclude watched or ineligible records, then retain the highest eligible score per ID. Equal scores for the same ID choose the smaller genre. Return at most k titles by descending score, breaking ties by ascending ID. Nonpositive k returns an empty result.

For `[(A,8), (B,9), (A,10), (C,9)]`, with B watched and k=2, output is `[(A,10), (C,9)]`. Ranking duplicates first could spend both result slots on A; filtering only after selection could return too few titles.

RANKING PIPELINE: EACH STAGE HAS ONE JOB



Eligibility and identity are resolved before choosing the best distinct titles.

Stage	State for this input
Input	`[(A,8), (B,9), (A,10), (C,9)]`
Exclude watched B	`[(A,8), (A,10), (C,9)]`
Resolve duplicate A	`[(A,10), (C,9)]`
Rank and limit	`[(A,10), (C,9)]`

These are original toy rules, not a claim about Netflix's actual recommendation formula. If eligibility belongs to a canonical catalog record rather than individual candidates, resolve that identity first; that changes the contract.

Go example: encode the complete ranking comparator

The function below ranks already distinct eligible candidates. It copies before sorting, preserving caller input.

```go
type Candidate struct {
    ID    string
    Score int64
}

func TopTitles(candidates []Candidate, k int) []Candidate {
    if k <= 0 {
        return nil
    }
    ranked := append([]Candidate(nil), candidates...)
    sort.Slice(ranked, func(i, j int) bool {
        a, b := ranked[i], ranked[j]
        if a.Score != b.Score {
            return a.Score > b.Score
        }
        // Resolve ties before truncating to k winners.
        return a.ID < b.ID
    })
    if k < len(ranked) {
        ranked = ranked[:k]
    }
    return ranked
}
```

For `[(D,7), (A,9), (C,7)]`, k=2, output is `[(A,9), (C,7)]`. The equal-score tie is decided before truncating, not afterward. k larger than input returns all; k<=0 returns empty. Sorting u candidates costs O(u log u), space O(u) for the copy.

Putting the program together: filter and deduplicate before ranking

```go
type CatalogCandidate struct {
    ID, Genre string
    Score     int64
    Eligible  bool
}

func RankEligible(input []CatalogCandidate,
    watched map[string]bool, k int) []Candidate {
    if k <= 0 {
        return nil
    }
    unique := make(map[string]CatalogCandidate)
    for _, item := range input {
        // Filter before an ID can occupy a result slot.
        if !item.Eligible || watched[item.ID] {
            continue
        }
        old, found := unique[item.ID]
        if !found || item.Score > old.Score ||
            (item.Score == old.Score &&
                item.Genre < old.Genre) {
            unique[item.ID] = item
        }
    }
    candidates := make([]Candidate, 0, len(unique))
    for _, item := range unique {
        candidates = append(candidates,
            Candidate{ID: item.ID, Score: item.Score})
    }
    return TopTitles(candidates, k)
}
```

For n input records and u distinct survivors, expected time is O(n+u log u), space O(u). A size-k heap can reduce selection to O(u log(k+1)), but the deduplication map still uses O(u) memory. If smaller IDs win ties, the heap root must consider the larger tied ID the worse winner.

Worked example: diversity needs candidates below the original cutoff

Problem. Choose k=2 distinct titles with at most one from each genre. Fill as many slots as possible up to k, then maximize total score for that count. Candidates are `[(A,action,10), (B,action,9), (C,drama,8)]`. Ordinary top 2 gives A and B, violating the rule. The constrained answer is A and C.

Candidate in score order	Chosen so far	Decision
A, action, 10	`[A]`	First action title
B, action, 9	`[A]`	Skip used genre
C, drama, 8	`[A, C]`	Fill second genre

With exactly one disjoint genre per title, sorting all distinct eligible candidates and accepting while genre quota permits maximizes score for the required count. This core assumes identity/eligibility have already been resolved. Negative scores still fill available slots under this contract; if selecting fewer titles is allowed solely to maximize total score, skip negative scores instead.

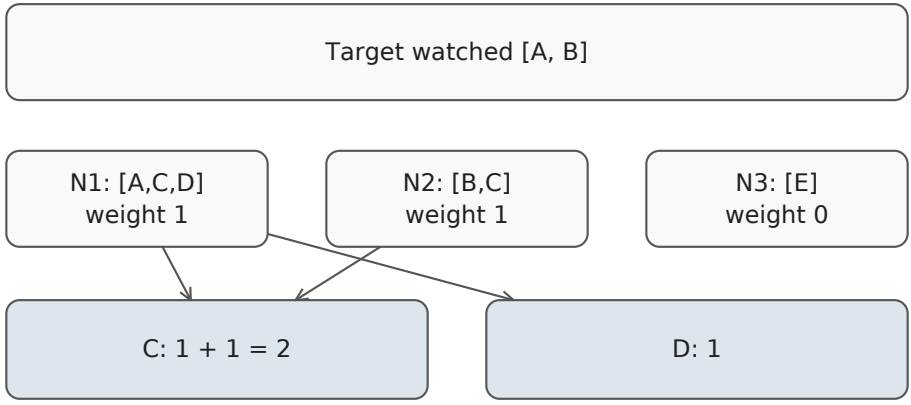
```go
func GenreWinners(input []CatalogCandidate,
    k int) []CatalogCandidate {
    if k <= 0 {
        return nil
    }
    ranked := append([]CatalogCandidate(nil), input...)
    sort.Slice(ranked, func(i, j int) bool {
        if ranked[i].Score != ranked[j].Score {
            return ranked[i].Score > ranked[j].Score
        }
        return ranked[i].ID < ranked[j].ID
    })
    used := make(map[string]bool)
    var out []CatalogCandidate
    for _, item := range ranked {
        if used[item.Genre] {
            continue
        }
        used[item.Genre] = true
        out = append(out, item)
        if len(out) == k {
            break
        }
    }
    return out
}
```

Discarding everything below ordinary top k would lose C before the constraint is applied. Multiple overlapping tags and simultaneous quotas need a new optimality argument; this simple greedy proof does not cover them. Interleaving source lists for fairness similarly differs from global score order. A heap over sorted source heads gives a k-way score merge; alternation gives a different policy.

Collaborative scoring: shared history becomes a weight

Problem. Target watched `[A, B]`. Neighbor histories are `N1=[A, C, D]`, `N2=[B, C]`, `N3=[E]`. Define similarity as the number of distinct shared titles. Add each neighbor's similarity to their unwatched titles.

SHARED TITLES BECOME NEIGHBOR WEIGHTS



Neighbors sharing A or B each contribute weight 1. C receives both contributions; D receives one.

Neighbor	Shared titles	Weight	Contributions
N1	`[A]`	1	C:+1, D:+1
N2	`[B]`	1	C:+1
N3	`[]`	0	None

Scores are `{C: 2, D: 1}`. Repeated views are deduplicated within each history so the formula counts distinct overlaps.

```go
func CollaborativeScores(target []string,
    neighbors [][]string) map[string]int64 {
    watched := make(map[string]bool)
    for _, id := range target {
        watched[id] = true
    }
    scores := make(map[string]int64)
    for _, history := range neighbors {
        distinct := make(map[string]bool)
        var weight int64
        for _, id := range history {
            // Repeated views count once toward similarity.
            if !distinct[id] && watched[id] {
                weight++
            }
            distinct[id] = true
        }
        if weight == 0 {
            continue
        }
        for id := range distinct {
            if !watched[id] {
                scores[id] += weight
            }
        }
    }
    return scores
}
```

Time is expected linear in supplied history size plus distinct-history scans. Storage includes watched, scores, and one neighbor's distinct set. An inverted index from title to users can narrow potential neighbors, while very popular titles still generate many matches. No overlap is a cold-start case; a fallback popularity list is another explicitly supplied rule.

Streaming averages: expiration must reverse contributions

Problem. Maintain per-title averages over `(now-W, now]`. Events carry title, event time, and positive duration. Input is monotone in event time. Store a sum and count per title and retain events so each expired contribution can be subtracted.

For A's durations `[10, 2]`, average is 6. When 10 expires, subtract it and decrement count: sum=2, count=1, average=2. A running mean alone does not preserve which old contribution to remove.

```go
type StatEvent struct {
    At      int64
    Title   string
    Minutes int64
}

type VideoTotal struct {
    Sum   int64
    Count int
}

type RollingStats struct {
    Window int64
    events []StatEvent
    totals map[string]VideoTotal
}

func (s *RollingStats) Expire(now int64) {
    expired := 0
    for expired < len(s.events) &&
        s.events[expired].At <= now-s.Window {
        e := s.events[expired]
        // Undo both parts of the average, not just the sum.
        total := s.totals[e.Title]
        total.Sum -= e.Minutes
        total.Count--
        if total.Count == 0 {
            delete(s.totals, e.Title)
        } else {
            s.totals[e.Title] = total
        }
        s.events[expired] = StatEvent{}
        expired++
    }
    s.events = s.events[expired:]
}
```

Clearing consumed records releases title references. A long-lived queue also needs compaction or a ring strategy to bound retained backing allocations. This core assumes Window>0 and fitting sums/counts.

```go
func (s *RollingStats) Record(e StatEvent) bool {
    if e.Minutes <= 0 {
        return false
    }
    s.Expire(e.At)
    if s.totals == nil {
        s.totals = make(map[string]VideoTotal)
    }
    total := s.totals[e.Title]
    total.Sum += e.Minutes
    total.Count++
    s.totals[e.Title] = total
    s.events = append(s.events, e)
    return true
}

func (s *RollingStats) Average(title string,
    now int64) (float64, bool) {
    s.Expire(now)
    total, found := s.totals[title]
    if !found {
        return 0, false
    }
    return float64(total.Sum) / float64(total.Count), true
}
```

All Record and query times must be nondecreasing. Otherwise a newly recorded old event can hide behind fresh events, and previously expired contributions cannot be restored. Each accepted event enters/leaves once, giving amortized O(1) updates; retained event memory depends on arrival rate times window width, not just number of titles.

Worked example: decreasing scores invalidate winner-only storage

Initially A totals 13 and B totals 6, so top 1 is A. An A contribution of 10 expires: scores become `{A: 3, B: 6}`, and top 1 must become B. A heap containing only the old winner A has forgotten its needed replacement.

Retain all live aggregates and sort on query for a clear baseline. An indexed heap or another ordered structure can maintain all changing scores. A static size-k selection proof is correct only while discarded candidates cannot become relevant later. Expiration changes that assumption; approximate heavy-hitter methods change the accuracy contract.

Chapter 11 – Modeling larger practical programs

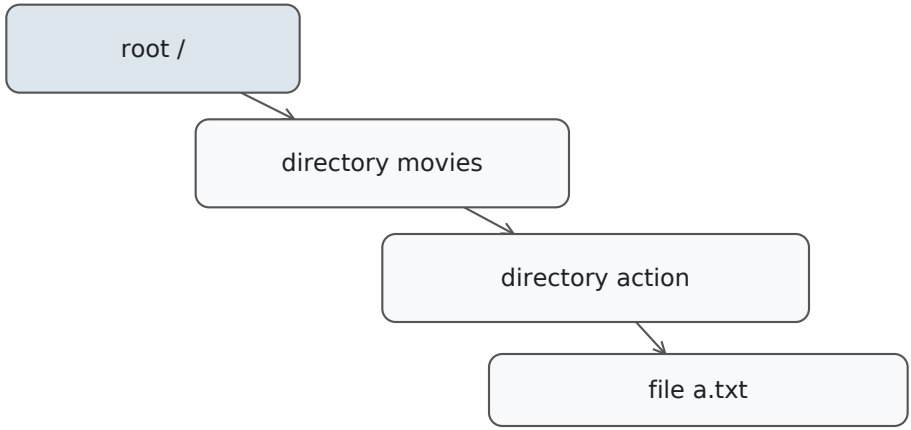
A larger exercise usually introduces several methods and shared invariants. This chapter models a filesystem, nested transactions, pub/sub delivery, and a cached playback service. For each, begin with the observable behavior and a small valid transition; the method names alone do not define the contract.

Filesystem: paths traverse named child maps

Problem. Support directories and file contents in memory. After creating `/movies/action` and writing `/movies/action/a.txt`, reading that path returns its contents and listing `/movies` returns `["action"]`.

Restrict the first version to normalized absolute paths without repeated separators, '.' or '..'. A file has contents; a directory has children. The tree represents ownership, not just path strings.

PATH COMPONENTS WALK A TREE



Path resolution follows root, movies, action, and the named file through child maps.

Operation	Required behavior
mkdir	Create directory; define missing-parent policy
write	Store contents; reject treating a directory as a file
read	Return file contents and presence/error
ls	List directory children, sorted if promised
delete	Define whether nonempty directories may be removed
move	Validate both ends before mutation; reject descendant cycles
copy	Create independent nodes, not shared child maps

Go example: resolve already validated path components

This is the path-resolution core, not a complete filesystem. The caller splits a normalized path into components such as `["movies", "action", "a.txt"]`, and the root is an existing directory.

```go
type FSNode struct {
    IsFile   bool
    Content  string
    Children map[string]*FSNode
}

func Resolve(root *FSNode,
    parts []string) (*FSNode, bool) {
    current := root
    for _, name := range parts {
        // A file cannot serve as an intermediate directory.
        if current == nil || current.IsFile {
            return nil, false
        }
        current = current.Children[name]
        if current == nil {
            return nil, false
        }
    }
    return current, current != nil
}
```

For the valid example, resolve movies, then action, then a.txt. A missing child or a file before the end fails. The empty component list resolves root. Work is expected O(depth) child lookups plus hashing/path parsing; this iterative core uses O(1) auxiliary state.

Moving `/movies` inside `/movies/action` would create a cycle. Validate ancestry, source existence, and destination collision before unlinking anything, so a rejected move preserves state. A deep copy must recreate child maps and nodes. Concurrency

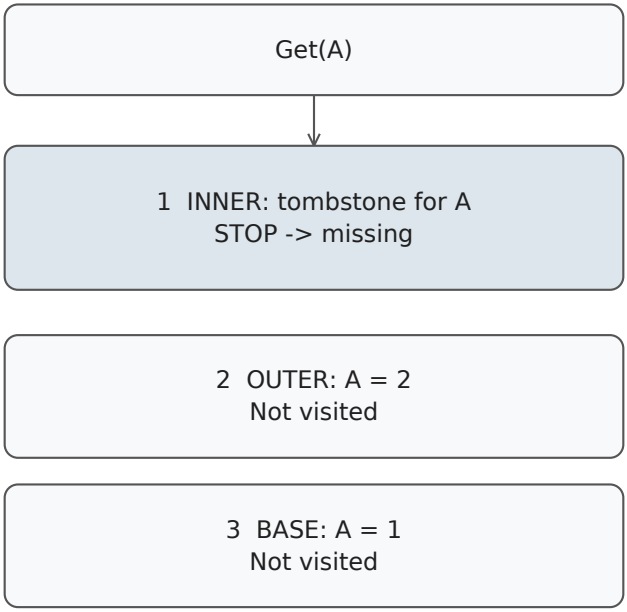
requires a lock around the complete multi-parent mutation, not independent locks around each map write.

Nested transactions: overlays contain instructions

Problem. Start with base `{A: "1"}`. Begin an outer transaction, set A to "2", begin an inner transaction, delete A, commit inner, then roll back outer. The final visible A must be "1".

A transaction overlay maps keys to edits. Reads search newest overlay first. An edit can mean a value, deletion, or no instruction. Only no instruction permits searching older layers. A deletion tombstone must stop lookup.

OVERLAY LOOKUP: DELETED IS NOT THE SAME AS ABSENT



A tombstone in the newest overlay hides lower values. It is different from an absent overlay entry.

Worked example: inner commit does not bypass outer rollback

Action	Base	Outer	Inner	Visible A
Initial	`{A:1}`	None	None	1
Begin; Set A=2	`{A:1}`	`{A:2}`	None	2
Begin; Delete A	`{A:1}`	`{A:2}`	`{A:deleted}`	Missing
Commit inner	`{A:1}`	`{A:deleted}`	None	Missing
Rollback outer	`{A:1}`	None	None	1

Committing inner merges into outer; it must not delete base A yet. Otherwise outer rollback could not restore the original value. Nested transactions here are single-session overlays, not a claim about concurrent database isolation.

Go example: layered reads distinguish absence and deletion

```go
type Edit struct {
    Value   string
    Deleted bool
}

func ReadLayered(base map[string]string,
    layers []map[string]Edit, key string) (string, bool) {
    for i := len(layers) - 1; i >= 0; i-- {
        edit, found := layers[i][key]
        if !found {
            continue
        }
        if edit.Deleted {
            // A tombstone hides every older value.
            return "", false
        }
        return edit.Value, true
    }
    value, found := base[key]
    return value, found
}
```

An edit with `Value: ""` and Deleted=false is a valid empty value. A missing map entry is no instruction. Deleting a key from an overlay would expose a lower value, so it cannot represent deleting the logical key.

Putting the program together: begin, commit, and rollback

The following complete transaction stack initializes base lazily. Begin pushes an empty overlay. Rollback discards only the newest overlay. Commit copies its edits to the parent, or applies them to base when no parent remains.

```go
type TransactionStore struct {
    base   map[string]string
    layers []map[string]Edit
}

func (s *TransactionStore) Begin() {
    s.layers = append(s.layers, make(map[string]Edit))
}

func (s *TransactionStore) Change(key string, edit Edit) {
    if len(s.layers) > 0 {
        s.layers[len(s.layers)-1][key] = edit
        return
    }
    if s.base == nil {
        s.base = make(map[string]string)
    }
    if edit.Deleted {
        delete(s.base, key)
    } else {
        s.base[key] = edit.Value
    }
}

func (s *TransactionStore) Rollback() bool {
    if len(s.layers) == 0 {
        return false
    }
    last := len(s.layers) - 1
    // Release the discarded map from the backing array.
    s.layers[last] = nil
    s.layers = s.layers[:last]
    return true
}
```

Commit reuses Change after removing the newest layer, so Change naturally targets its parent or base.

```go
func (s *TransactionStore) Commit() bool {
    if len(s.layers) == 0 {
        return false
    }
    last := len(s.layers) - 1
    top := s.layers[last]
    s.layers[last] = nil
    s.layers = s.layers[:last]
    // Change now targets the parent overlay or the base.
    for key, edit := range top {
        s.Change(key, edit)
    }
    return true
}

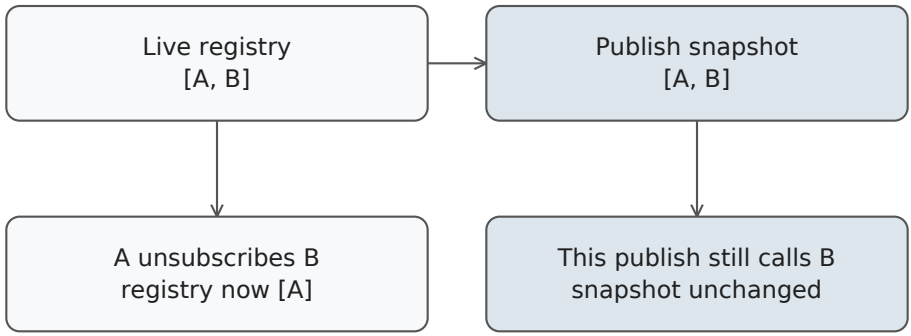
func (s *TransactionStore) Get(key string) (string, bool) {
    return ReadLayered(s.base, s.layers, key)
}
```

A read costs expected O(transaction depth); commit costs O(changes in the top layer). Rollback discards a layer in O(1), excluding later garbage collection. Clearing the removed slice slot prevents its backing array from retaining the overlay map. Call Commit or Rollback without an active transaction and receive false, with no mutation. Concurrency needs an explicit session/isolation contract before adding locks around this shared stack.

Pub/sub: callbacks need a delivery snapshot

Problem. Subscribers A and B receive a message. During A's callback, A unsubscribes B. Under snapshot semantics, B still receives this publication because it was subscribed at the start; the next publication excludes B.

COPY MEMBERSHIP, THEN UNLOCK AND CALL



Publishing copies callback membership under the lock and invokes the snapshot after unlocking.

A topic maps subscriber IDs to callbacks. Return an ID from Subscribe: arbitrary Go function values cannot be equality-compared to identify subscriptions. Calling callbacks while holding the registry lock can deadlock when a callback subscribes or unsubscribes.

Worked example: live registry and current snapshot differ

Event	Live subscribers	Publish snapshot
Publish begins	`[A, B]`	`[A, B]`
A unsubscribes B	`[A]`	`[A, B]`
B is called	`[A]`	`[A, B]`
Next publish	`[A]`	`[A]`

The core below accepts a prebuilt registry and promises no callback order. Mutation methods must use the same mutex.

```go
type TopicBus struct {
    mu        sync.Mutex
    callbacks map[string]map[int]func(string)
}

func (b *TopicBus) Publish(topic, message string) {
    b.mu.Lock()
    var snapshot []func(string)
    for _, callback := range b.callbacks[topic] {
        snapshot = append(snapshot, callback)
    }
    b.mu.Unlock()
    // Callbacks may safely change subscriptions after unlock.
    for _, callback := range snapshot {
        callback(message)
    }
}
```

The lock protects membership copying; it does not cover user callbacks. Cost is O(s) for s subscribers plus their work, with O(s) snapshot space. This synchronous core lets one slow callback delay later ones and lets a panic propagate. Asynchronous delivery adds queue bounds, cancellation, ordering, and an overflow policy; unbounded goroutines do not solve backpressure.

Playback service: compose the contracts already defined

Problem. `GetPlayback(user, video)` uses an expensive metadata backend. Cache fresh metadata, share concurrent loads, bound cache space, and report errors and hit/miss metrics. Authorization remains a separate per-user check.

Step	Decision and state change
Validate/authorize	Confirm this user may request this video
Fresh cache lookup	Hit returns metadata; miss continues
Join/create load	One leader fetches; followers wait
Backend response	Share success or error with every waiter
Admission	Successful oversized metadata may be returned without caching
Metrics	Count request hit/miss separately from backend loads

If 3 requests miss A and share one fetch, record 3 misses and 1 backend load. Admission rejection is not backend failure. If stale data may be served during an outage, specify its freshness limit and measure that outcome separately.

The cache key must cover the metadata's identity. Do not cache a user-specific authorization decision under video ID alone. A late response after invalidation may require a version check before admission. Cancellation of one caller need not cancel shared work still needed by other callers.

Start with the smallest stated behavior, then name the additional state for each follow-up. A service composition is understandable when every method preserves the local contracts rather than hiding them behind one large function.

Chapter 12 - Complete interview case studies

These cases assemble the earlier techniques into complete programs. Each starts with a brief and a concrete trace, then shows code in dependency order. The first implements an LRU cache with TTL using a simple correct cleanup baseline. The second filters, deduplicates, aggregates, and ranks recent viewing events. You can follow both without executing anything.

Case one: fresh metadata with bounded recency

Brief. Implement a single-threaded metadata cache with Put(key, value, ttl) and Get(key). Capacity limits resident entries. Never return an expired value. Reads and writes promote live entries. Before evicting a live LRU entry, remove expired residents. Inject the clock.

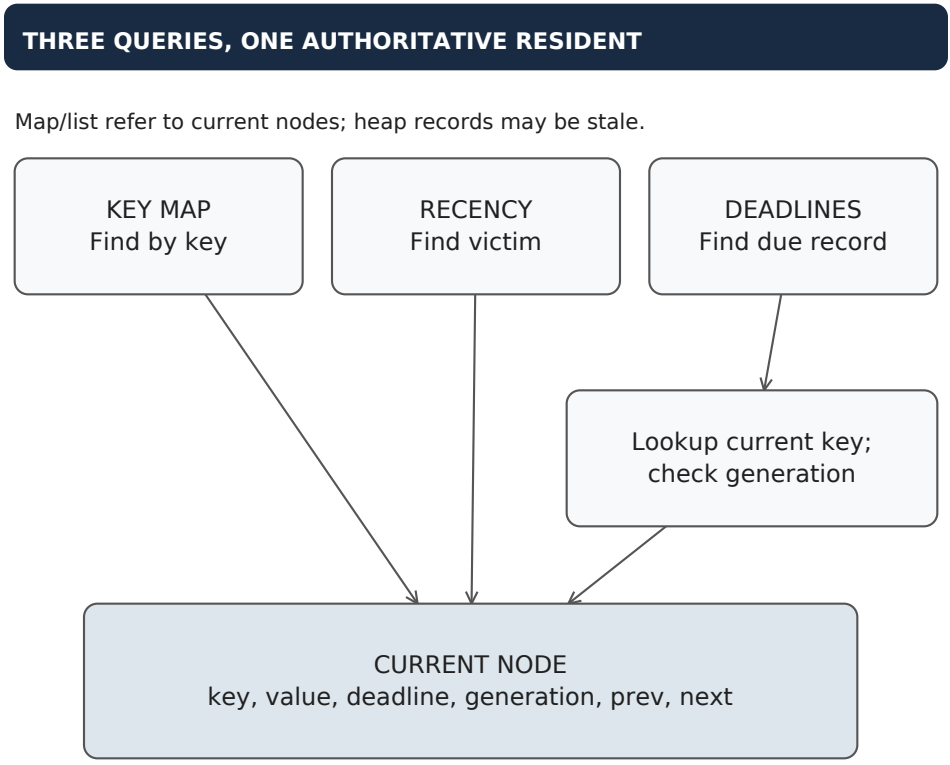
Choose exact rules: capacity<=0 stores nothing; `now >= deadline` means expired; ttl<=0 removes the key; an overwrite resets deadline and promotes; Get does not extend TTL; empty strings are valid values. Tick arithmetic must fit int64.

Worked example: expiration wins over recent access

Use capacity 2 and begin at time 0. Put A with TTL 5, B with TTL 20, and Get A. A is now most recent, but it still expires at 5.

Time	Operation	MRU to LRU	Deadlines
0	Put A="HD", TTL 5	`[A]`	`{A:5}`
0	Put B="UHD", TTL 20	`[B, A]`	`{A:5, B:20}`
0	Get A	`[A, B]`	Unchanged
5	Put C="HDR", TTL 20	`[C, B]`	`{B:20, C:25}`
6	Put C="new", TTL 50	`[C, B]`	`{B:20, C:56}`
25	Get C	`[C, B]`	Return `("new", true)`

At time 5, removing only the LRU tail B would violate the expired-first rule. A is expired at the front. Freshness and recency have different orderings.



Identity, recency, and deadline indexes answer different questions about the same cache entries.

The diagram shows the heap-based optimization from chapter 7. The complete implementation below deliberately uses a map scan for cleanup, so its correctness does not depend on a third index. That scan is the operation we would later optimize.

Go example: state and constructor

A node has stable identity, value, deadline, and two list neighbors. The map and list must describe the same residents. Sentinels are never residents.

```go
type metadataNode struct {
    key, value string
    deadline   int64
    prev, next *metadataNode
}

type MetadataCache struct {
    capacity   int
    clock      func() int64
    byKey      map[string]*metadataNode
    head, tail *metadataNode
}

func NewMetadataCache(capacity int,
    clock func() int64) *MetadataCache {
    if capacity < 0 {
        capacity = 0
    }
    head, tail := &metadataNode{}, &metadataNode{}
    head.next, tail.prev = tail, head
    return &MetadataCache{
        capacity: capacity, clock: clock,
        byKey: make(map[string]*metadataNode),
        head:  head, tail: tail,
    }
}
```

The clock must be non-nil and monotone. No mutex is present; callers serialize operations. This is a complete count-and-TTL baseline, not a distributed cache.

Helpers update one invariant at a time

Unlink updates only neighbor links. Promote unlinks an already resident node before inserting it at the front. Remove updates both list and map.

```go
func unlinkMetadata(n *metadataNode) {
    n.prev.next = n.next
    n.next.prev = n.prev
}

func (c *MetadataCache) front(n *metadataNode) {
    first := c.head.next
    n.prev, n.next = c.head, first
    c.head.next, first.prev = n, n
}

func (c *MetadataCache) remove(n *metadataNode) {
    unlinkMetadata(n)
    delete(c.byKey, n.key)
}

func (c *MetadataCache) expire(now int64) {
    for _, n := range c.byKey {
        if n.deadline <= now {
            c.remove(n)
        }
    }
}
```

Deleting the current map entry during Go map iteration is allowed. The scan may remove several list positions; stable node pointers make that safe. Each removal preserves the neighbors' connection. Removed nodes are no longer used, so clearing their own links is optional for this implementation.

Get: check freshness before promoting

```go
func (c *MetadataCache) Get(key string) (string, bool) {
    n, found := c.byKey[key]
    if !found {
        return "", false
    }
    if c.clock() >= n.deadline {
        // Expiration takes precedence over recency promotion.
        c.remove(n)
        return "", false
    }
    unlinkMetadata(n)
    c.front(n)
    return n.value, true
}
```

A hit promotes without changing deadline. A miss does not change recency; an expired lookup removes that one resident. Expected time is O(1), and an empty value still returns true.

Put: cleanup before live capacity eviction

```go
func (c *MetadataCache) Put(key, value string,
    ttl int64) {
    if ttl <= 0 {
        if n, found := c.byKey[key]; found {
            c.remove(n)
        }
        return
    }
    if c.capacity == 0 {
        return
    }
    now := c.clock()
    // Reclaim expired residents before evicting a live one.
    c.expire(now)
    n, found := c.byKey[key]
    if found {
        unlinkMetadata(n)
    } else {
        n = &metadataNode{key: key}
        c.byKey[key] = n
    }
    n.value, n.deadline = value, now+ttl
    c.front(n)
    if len(c.byKey) > c.capacity {
        c.remove(c.tail.prev)
    }
}
```

At time 5, expire removes A regardless of its recency, then C fits beside B. An overwrite reuses its node. Completed Put satisfies count<=capacity and no resident was expired at its sampled now. The map/list correspondence remains one-to-one.

The baseline Put is O(n) because it scans residents. Space is O(capacity), Get expected O(1). A deadline heap changes the cleanup cost to due-record work plus heap updates; generations must guard stale records. A used-weight counter

changes count capacity into weighted capacity and may require several live evictions.

Checking the complete cache

A controllable clock makes these tests concrete:

Input/transition	Required outcome
Capacity 0; Put A	Always missing
Put A="" with live TTL	`("", true)`
Get at exact deadline	Missing; unlink and delete
Refresh before old deadline	New deadline governs reads
Repeated Put A	One A node only
Expired MRU; insert C	Expired A leaves before live B
Hit already-first node	List remains connected

For concurrency, wrap each complete method in one exclusive lock. For backend loading, use chapter 7's SharedLoader outside the cache lock and recheck freshness inside shared work. Invalidation during a load needs a version policy. These follow-ups add explicit guarantees; they do not change the meaning of TTL silently.

Case two: recent viewing totals and top titles

Brief. Given a batch of events, return the top k titles by total recent watch minutes. Event ID deduplicates redelivery. Include event times in `(now-window, now]`; exclude future events. Rank descending total, then ascending title ID. Input order is arbitrary and preserved.

Repeated IDs must have identical payloads. Durations are positive, sums and window-boundary arithmetic fit int64, and window>0. The public pipeline rejects invalid duration/window input. The event type omits user because this exercise ranks the supplied cohort; per-user ranking would require grouping or filtering by user too.

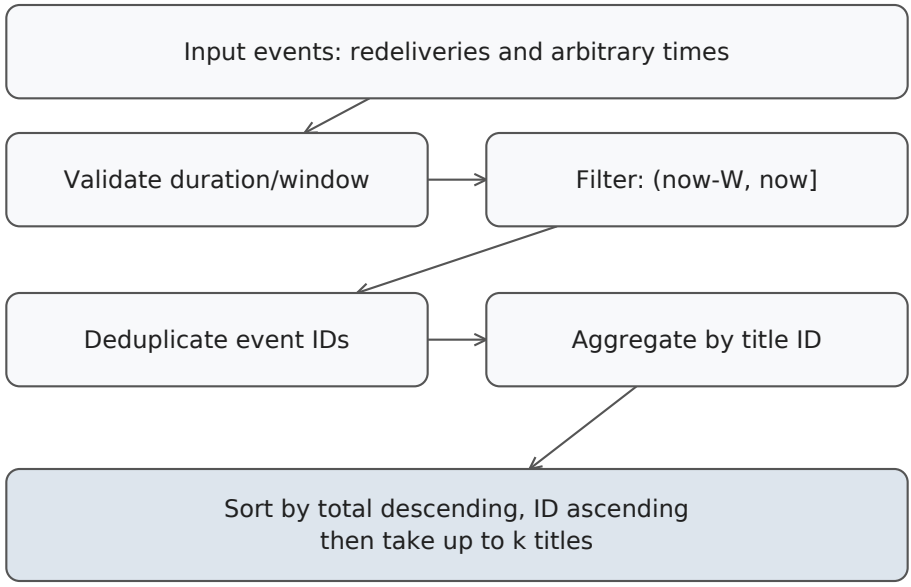
Worked example: filter, deduplicate, then aggregate

Set now=10, window=5, k=2:

Event ID	Title	Time	Minutes	Contribution
e1	A	6	3	A:+3
e2	A	9	4	A:+4
e2 again	A	9	4	Duplicate; +0
e3	B	10	7	B:+7
e4	C	5	100	Exact left boundary; +0
e5	D	11	20	Future; +0

Totals are `{A: 7, B: 7}`. A wins the ID tie, so the result is `[(A, 7), (B, 7)]`. Filtering before deduplication is safe only because identical IDs carry identical timestamps and payloads. Conflicting redeliveries would need a canonicalization or rejection rule.

RECENT RANKING: EACH STAGE HAS A CONTRACT



The batch pipeline validates, filters by time, resolves event identity, totals by title, and ranks the output.

Go example: preserve identity and aggregate contributions

```go
type WatchEvent struct {
    ID, Title   string
    At, Minutes int64
}

func RecentTotals(events []WatchEvent,
    now, window int64) map[string]int64 {
    seen := make(map[string]bool)
    totals := make(map[string]int64)
    for _, event := range events {
        // Exclude the left boundary and all future events.
        if event.At <= now-window || event.At > now {
            continue
        }
        if seen[event.ID] {
            continue
        }
        // Deduplicate deliveries by event ID, not title ID.
        seen[event.ID] = true
        totals[event.Title] += event.Minutes
    }
    return totals
}
```

This helper assumes validated positive durations and window. It does not mutate input. Each relevant logical event contributes exactly once because its ID is recorded before another delivery can add it. The set is by event ID; the totals map is by title ID. Confusing those keys either drops distinct views or double-counts redelivery.

Putting the program together: validate and use the shared comparator

```go
func RecentRanking(events []WatchEvent,
    now, window int64, k int) ([]Candidate, bool) {
    if window <= 0 {
        return nil, false
    }
    // Validate the whole batch even when k is nonpositive.
    for _, event := range events {
        if event.Minutes <= 0 {
            return nil, false
        }
    }
    if k <= 0 {
        return nil, true
    }
    totals := RecentTotals(events, now, window)
    candidates := make([]Candidate, 0, len(totals))
    for title, minutes := range totals {
        candidates = append(candidates,
            Candidate{ID: title, Score: minutes})
    }
    return TopTitles(candidates, k), true
}
```

TopTitles and Candidate come from chapter 10; their definitions are part of the same example package. The boolean here reports valid input, not whether an event exists. Validation runs even for k<=0 under this contract. An empty valid batch returns an empty ranking; k larger than available titles returns all.

For n deliveries and u live titles, expected scan work is O(n), sorting O(u log u), and auxiliary state O(n+u). A k-winner heap reduces selection work but does not eliminate the deduplication set or totals map.

Turning the batch into a stream

A batch can scan arbitrary order, but a FIFO streaming expiry queue requires monotone event times. Keep each accepted contribution `(ID, title, time, minutes)` so expiration can subtract it from the title's aggregate. Deduplication retention must cover the same query horizon or another explicitly defined horizon.

Worked example: remove the contribution before reranking

At now=9, width=10, A has events `(time 0, minutes 10)` and `(time 9, minutes 3)`, while B has `(time 8, minutes 6)`. Totals are `{A: 13, B: 6}` and top 1 is A. At now=10, A's time-0 event reaches the excluded boundary and expires.

State	A total	B total	Top 1
Before expiry	13	6	A
Subtract A's old 10	3	6	B

Chapter 10's RollingStats retains the contribution to reverse. Ranking all current aggregates is a simple exact baseline. A winner-only heap cannot recover B if it discarded B earlier; dynamic score decreases require retaining outsiders too.

Both cases follow one method: state the result and boundaries, build a correct baseline, show every mutation on a tiny input, then identify the specific expensive step to optimize. The code and trace must agree on the same contract before any complexity improvement is useful.

Chapter 13 Hiring manager interview preparation

This chapter prepares you for the manager conversation after your first DSA round. The target is **Senior Software Engineer 5, Developer Platform, Warsaw**, requisition **JR32657**. Prepare to show how you turn an engineering problem into a useful, reliable product, make decisions, and work with the people affected by those decisions. Your own contribution and the consequences of your choices should be clear in every major story.

Research checked on **13 October 2026**. The recruiter brief below is specific to your interview. The questions and assessment guidance are preparation recommendations, not Netflix's internal scorecard or a verified question bank. The posting covers several teams; the exact team and meeting duration remain unspecified.

The recruiter brief

The guidance you received explicitly names experience and projects, technology stack, and interest in Netflix. It recommends revisiting the culture memo, understanding the position, and discussing projects you enjoyed that involved complexity, with examples of your contribution and involvement. There will also be room for your questions.

Our preparation order should therefore be: two projects in depth; your motivation for this role; feedback and disagreement stories; independent decisions and setbacks; then questions for the manager. This order follows your brief, not an assumed universal interview sequence.

What this particular role changes

The supplied posting concerns software for engineers, with independent delivery, design writing, customer and product collaboration, reliability, testing, data modelling, and work across the stack. Its stated scale includes hundreds of users and thousands of requests per minute. It mentions Java, Python and Go; the current Workday version of the same requisition emphasizes Java/Python, AWS, containers and CI/CD. A backend specialty is compatible with broader familiarity. Supplied role, same requisition on Workday.

The current Workday description lists Availability Tooling, Knowledge Platform, Health Integrity of the Paved Road, and Change Systems. These span operational visibility, developer answers and documentation, ecosystem health, and large-scale change automation. The supplied page and Workday have different detail, so establish which team's charter and stack this manager owns. Current team descriptions.

Preparation implications. Treat developers as customers. A project becomes more relevant when you can explain the workflow it improved, why people adopted it, how you supported it, and what evidence showed that it helped. Complexity can come from dependencies, incompatible requirements, rollout risk, or coordination; it does not require enormous request volume. Using Go in the coding interview does not establish the production stack.

Netflix's engineering writing provides useful context. A 2018 article describes teams owning software through operation and support, aided by shared tools whose usefulness encourages adoption. This is historical context, not a promise about every team's current operating model. A 2026 migration account illustrates preserving customer contracts, validating real workloads, observing rollout progress, and retaining a fallback. Full Cycle Developers, Cassandra Data Movement.

What the interview research supports

Several kinds of evidence point toward preparing both project detail and working style:

- A software-engineer report dated February 2025 describes project and technology probing by a manager, and a later discussion of the memo. It is an anonymous candidate account, with no verified level. Candidate report.

- A January 2026 frontend community account reports discussion of a proud project, friction with an engineer, and repeated negative feedback from a manager. Its frontend context limits what it says about your technical interview. Community accounts.

- A public L5 mock with a Netflix engineer rewards customer understanding, collaboration across teams, business impact, self-awareness and precise personal contribution. Its feedback identifies excessive technical detail and general answers without an actual event as weaknesses. A mock demonstrates one interviewer's coaching, not an official hiring rubric. Mock and written feedback.

- An SRE candidate describes a manager conversation combining operational experience and culture. A Warsaw full-stack candidate reports different sequences with different teams, ultimately receiving an L4 offer. These support variation, not a prescribed L5 schedule. SRE account, Warsaw account.

- An L5 Ads account describes ambiguity, risk, product influence and conflict questions, with comparatively little memo discussion. That is evidence about another domain; your recruiter's explicit culture guidance takes precedence. Ads candidate account.

These accounts cannot establish question frequencies, pass thresholds, or why someone was rejected. Their useful common

ground is concrete experience, decision quality and collaboration. Treat claims such as a fixed percentage of culture questions, a universal loop, or guaranteed rejection for one phrase as unsupported rules.

Culture expectations to understand

Netflix's 2024 revision organizes its culture around four principles: **The Dream Team**, **People over Process**, **Uncomfortably Exciting**, and **Great and Always Better**. Together, they emphasize effective colleagues, responsible autonomy, willingness to adapt and experiment, and continuing improvement. The revision explicitly distinguishes useful process from unnecessary process. Netflix's explanation of the revision.

The current memo names selflessness, judgment, candor, creativity, courage, inclusion, curiosity and resilience. It expects a decision owner to seek opposing views, make the call, and examine the outcome; colleagues then support execution. Managers provide context and coaching. Respect remains essential when exchanging criticism. Performance is considered across someone's record, including room for unsuccessful bets; regular conversations should prevent surprises. Its “keeper test” concerns whether a manager would retain or rehire someone. The memo also asks employees to support a broad range of creative work, potentially including content they personally dislike. Current culture memo.

The practical preparation task is to find real situations that expose your reasoning. A mistake you recognized and corrected can demonstrate more than an idealized success. Have a considered view of the memo, including an aspect you would want to understand in practice. Netflix's official new-graduate guidance explicitly invites discussion of aspects candidates do and do not connect with; that advice is useful,

although its interview sequence does not describe L5 hiring. Official candidate guidance.

Likely question families and strong answer ingredients

The wording below is original practice material. “The manager may look for” describes our interpretation of the evidence and this role, not access to an internal evaluation form. Q01-Q06 follow the recruiter brief most directly; Q07-Q18 broaden coverage of the working behaviours that can surface while discussing those projects.

Q01 Your career and current scope

Possible question: Walk me through your experience and what you are responsible for today.

A strong answer contains: Your current engineering focus, the users or business you serve, the scope you own, one or two relevant achievements, and why this opportunity follows naturally. Explain important career moves accurately. Select detail that helps the listener place your experience.

The manager may look for: A coherent account of your seniority and relevance. Make it possible to distinguish implementation, technical leadership and people management.

Follow-ups: Which decisions are yours? Who else works on the system? Why are you considering a move now?

Weakness to avoid: A chronological list of employers and technologies with no clear contribution or direction. Do not rewrite an involuntary move as a strategic choice.

Q02 Why Netflix and why Developer Platform

Possible question: What interests you about Netflix and this position?

A strong answer contains: A specific class of developer problems you enjoy, an example showing that interest, the connection to the advertised work, and a considered view of the working environment. Explain what you want to contribute and learn. Distinguish the shared platform role from any team assignment that remains unconfirmed.

The manager may look for: Informed motivation and a realistic match between your interests and the daily work.

Follow-ups: Why internal tools? What would you want to improve for engineers? Which part of this role stretches you?

Weakness to avoid: Depending entirely on brand prestige, compensation, favourite shows, or generic claims about scale. Those interests need not be hidden; they do not explain your fit for this job.

Q03 A project you enjoyed that involved complexity

Possible question: Tell me about a project you enjoyed. What made it difficult, and what did you personally do?

A strong answer contains: The users and problem; stakes and constraints; your exact responsibility; the hardest decision; alternatives; implementation and coordination; rollout; observable results; and what you would change. Explain what you enjoyed about the work itself.

The manager may look for: Ownership that survives detailed follow-up, technical depth, and connection between engineering work and a useful outcome.

Follow-ups: What was already decided? What did someone else own? Where did you disagree? What broke? What happened after launch?

Weakness to avoid: A tour of the architecture that never exposes a difficult decision. Make both individual contribution and other people's credit visible.

Q04 Technology choices and depth

Possible question: Why did you choose that language, database or architecture? What were its limitations?

A strong answer contains: The actual requirements, plausible alternatives, selection criteria, an explicit cost of your choice, and an example of operating or debugging it. Separate technologies you selected from ones you inherited. For unfamiliar parts of the advertised stack, describe your actual exposure and a concrete example of learning an adjacent technology.

The manager may look for: Technical judgment and accurate self-assessment, including the ability to work beyond a favourite tool.

Follow-ups: When would the rejected option be better? What bottleneck did you measure? What would cause you to reconsider?

Weakness to avoid: Treating technology names as proof of expertise, or promising that every language or database is interchangeable.

Q05 Developer needs and adoption

Possible question: How did you know engineers needed this tool, and how did you get them to use it?

A strong answer contains: A concrete workflow, how you learned about its pain, the initial user group, competing options, usability or migration friction, feedback that changed the product, and evidence of sustained use. Explain when keeping a local solution was reasonable.

The manager may look for: Product thinking in an internal platform role: understanding the cost to the customer, not only the elegance of your implementation.

Follow-ups: Who resisted and why? What did you remove from the plan? What if adoption stayed low?

Weakness to avoid: Equating “we launched it” with “it helped,” or portraying reluctant customers as the problem.

Q06 Outcomes and measurement

Possible question: How did you know the project succeeded?

A strong answer contains: A baseline, a relevant measure, the observation period and population, and limits on attributing the change to your work. Depending on the project, useful evidence might concern setup effort, time waiting for builds, failed changes, support demand, task completion, or cost per successful operation.

The manager may look for: A defensible link between a technical change and customer value, plus awareness that one metric can mislead.

Follow-ups: Did quality worsen? Were slow users excluded? Was usage mandated? How did you estimate time saved?

Weakness to avoid: Invented precision, an unexplained percentage, or treating fewer tickets as success when users may simply have stopped asking for help. Qualitative evidence is acceptable when described honestly.

Q07 Independent decisions under ambiguity

Possible question: Describe a situation where the goal was clear but nobody had specified what to build.

A strong answer contains: What was unknown, how you obtained useful context, the options you considered, whom you consulted, the decision you owned, and how you limited or monitored the risk. Identify what would have justified escalation or a change of course.

The manager may look for: Initiative with a sound decision process, rather than dependence on detailed instructions.

Follow-ups: What did you do before asking for help? What information was worth waiting for? Who could reverse your decision?

Weakness to avoid: Presenting approval collection as your main achievement, or implying that independence meant surprising affected teams. Describe your real authority at the time.

Q08 Disagreement with an engineer or manager

Possible question: Tell me about a significant technical disagreement. How did you handle it?

A strong answer contains: The other person's strongest argument, the shared objective, your concern, the evidence or experiment used, the decision process and the effect on the working relationship. Include your own contribution to the friction if relevant.

The manager may look for: Constructive challenge, curiosity and the ability to resolve disagreement without turning it into a contest.

Follow-ups: What would that person say? Did you speak to them directly? What changed your mind? Was escalation necessary?

Weakness to avoid: A story in which the other person is foolish and you inevitably win. A civil conversation alone is not evidence that the underlying problem was resolved.

Q09 Supporting a decision you opposed

Possible question: What happened when a decision went against your recommendation?

A strong answer contains: Your objection, how it was heard, who had authority to decide, what you did to make the chosen approach succeed, and what later evidence justified revisiting it, if any.

The manager may look for: The ability to challenge a proposal and then contribute reliably to execution.

Follow-ups: Did you continue lobbying? What did you learn from the result? What would have made compliance inappropriate?

Weakness to avoid: Passive compliance followed by “I told you so,” or suggesting that a final decision removes the duty to raise new material risks. Explain any safety or ethical boundary in the actual situation.

Q10 Difficult feedback you received

Possible question: What is a meaningful piece of critical feedback you received, and what changed afterward?

A strong answer contains: The specific behaviour, why it affected others, your initial reaction, how you checked your understanding, the action you took, and evidence from a later situation. If the feedback was partly wrong, separate its useful part from the part you challenged.

The manager may look for: Self-awareness and observable learning, rather than a claim that you welcome criticism.

Follow-ups: Was the feedback repeated? What would your manager still ask you to improve? How do you know your change worked?

Weakness to avoid: A disguised compliment, a trivial weakness, or a story that spends most of its energy discrediting the feedback giver.

Q11 Difficult feedback you gave

Possible question: Describe a conversation where you had to tell a colleague something uncomfortable.

A strong answer contains: An observable behaviour and its effect, why you raised it, how you chose the setting, what you said in substance, how you listened, and what happened afterward. Clarify relevant reporting or project relationships so direct communication does not sound like an unexplained escalation.

The manager may look for: Useful directness, empathy and follow-through.

Follow-ups: How did they respond? What if they disagreed? Did you later check whether things improved?

Weakness to avoid: Equating honesty with bluntness, diagnosing someone's personality, or adding an imaginary manager conversation to make the story sound better.

Q12 Failure and operational responsibility

Possible question: Tell me about a decision or release of yours that went wrong.

A strong answer contains: Your actual mistake, its consequences, how you communicated it, mitigation, investigation, and a durable improvement. Explain what evidence you had at the time and what you missed. A poor product bet can work as well as a production incident.

The manager may look for: Accountability, composure, and learning that changes later behaviour.

Follow-ups: Who was affected? Why did tests or review miss it? What was your role in recovery? How did you verify the fix?

Weakness to avoid: A failure that was entirely someone else's fault, a heroic recovery with no prevention, or an invented outage chosen because it sounds senior.

Q13 Influencing a migration across teams

Possible question: How have you moved several teams to a new API, tool or platform when you did not manage them?

A strong answer contains: Who benefited, who paid the migration cost, dependencies, compatibility requirements, early users, rollout and reversal options, support arrangements, and how you handled exceptions or resistance.

The manager may look for: Influence without formal authority and attention to the customer's ability to change safely.

Follow-ups: Which team was hardest to move? What commitment did you make? How did you know migration was complete? Who owned the old system afterward?

Weakness to avoid: Making the announcement or mandate the whole story. Completion includes the user's transition and the cost of maintaining what remains.

Q14 Prioritization and company interests

Possible question: Tell me about choosing between competing requests, or stopping work you had invested in.

A strong answer contains: The competing benefits and costs, constraints, who would lose out, the choice you made, how you communicated it, and evidence that would reopen the decision. Include a case where a wider benefit outweighed your team's preferred plan, if you have one.

The manager may look for: Deliberate allocation of effort and willingness to change direction despite sunk cost or personal attachment.

Follow-ups: What did you explicitly stop? Who disagreed? What would happen if you did nothing?

Weakness to avoid: Claiming to deliver everything through extra hours, or describing persistence without asking whether the goal remained worthwhile.

Q15 Inclusion and helping colleagues succeed

Possible question: Describe a time another person's perspective changed the outcome, or you helped someone become more effective.

A strong answer contains: The contribution that was being missed, what you did to make participation possible, what changed in the decision or capability, and appropriate credit. Examples can concern remote colleagues, experience differences, onboarding or communication styles.

The manager may look for: The ability to improve collective results and reconsider your assumptions.

Follow-ups: What did you learn from them? Could they operate independently afterward? Whose view was still missing?

Weakness to avoid: “I treat everyone the same” without an event, or a mentoring story where you repeatedly rescued someone without transferring understanding.

Q16 Learning and adapting to change

Possible question: When did a new technology, requirement or discovery force you to change your approach?

A strong answer contains: The gap you recognized, your learning or experiment, who helped, evidence that overturned an assumption, and what you delivered or stopped as a result. Show what remained uncertain.

The manager may look for: Adaptability with enough discipline to distinguish a useful new approach from novelty.

Follow-ups: How did you validate your understanding? What did you deliberately not learn or build? How did you protect users while experimenting?

Weakness to avoid: Treating a tutorial as production expertise, or implying that enthusiasm for GenAI proves the quality of an AI feature.

Q17 Your view of the culture memo

Possible question: Which parts of the memo connect with your experience, and which would be challenging?

A strong answer contains: A specific idea, a real event that informs your opinion, and a thoughtful question about how this team applies it. For performance expectations, explain how you seek clarity and feedback. For creative-content questions, express your actual position and boundaries honestly.

The manager may look for: Informed self-assessment and the ability to have a candid professional conversation.

Follow-ups: When have you worked differently? What would you need to adjust? What concern would you want resolved before joining?

Weakness to avoid: Reciting slogans, claiming perfect agreement, or manufacturing a provocative objection. Do not pretend comfort with conditions that would make the job unsuitable for you.

Q18 Strengths, growth areas and your manager

Possible question: What would your previous manager say about you? What support helps you do your best work?

A strong answer contains: A strength with evidence, a material growth area with recent action, and the context or

feedback you find useful. Explain how you work independently while keeping others informed. Connect future growth to the work you want to do.

The manager may look for: Accurate expectations, realistic self-knowledge and a workable partnership.

Follow-ups: What feedback keeps recurring? When do you ask for help? How would you approach your first months here?

Weakness to avoid: Saying you need no support, inventing a spotless reference, or promising an immediate rewrite before understanding the team.

Additional prompts for the team behind this opening

These are original discussion prompts suggested by the possible charters, not reported questions. Use the branch relevant to the manager's team; do not assume every branch is required.

Possible focus	Useful questions to rehearse
Operational tooling	How did you identify actionable signals in noisy telemetry? What happens when a dependency map is incomplete? How did a tool change an engineer's response to an incident?
Developer knowledge	How do you know an answer is correct and current? What happens when a document is stale? If AI is involved, how do you evaluate unsupported answers, permissions, escalation and user trust?
Ecosystem health	How would you choose a health indicator that teams can act on? How do you avoid a score that rewards superficial compliance? How do you make an upgrade easier for a busy team?
Change automation	How do you constrain the impact of a bad change? What makes retries safe? How do you pause, resume or reverse a campaign and explain partial progress to its owners?

For a question outside your direct experience, state the gap, identify the closest genuine example, and explain your reasoning. Keep hypothetical plans separate from things you have done.

Build answers around evidence

Prepare for a technical project deep dive

For each of your two main projects, keep enough detail to follow a request or workflow through the system. These are preparation prompts, not a requirement to turn every answer into a design interview.

- **Requirements and architecture:** Users, constraints, a representative operation, component boundaries and the choice that made the design difficult.

- **Data and interfaces:** The data model, important access patterns, consistency or compatibility assumptions, and how the API or interface exposed failures to users.

- **Scale and performance:** Actual workload, the measured bottleneck, how you investigated it, what improved, and what the next limit would be. Distinguish requests per minute from requests per second.

- **Testing and delivery:** A difficult failure mode, why the existing tests were insufficient, the validation you added, and how builds or delivery checks supported confidence.

- **Operation and change:** Signals you monitored, incident or support experience, rollout controls, recovery, and ownership after launch.

- **Stack breadth:** What you personally built or debugged in the backend, storage, delivery tooling and user interface. State where you worked with a specialist rather than claiming their expertise.

If a project was unfinished, say what was delivered, what was only designed and what result had not yet been measured. If a detail is confidential, explain the decision using safe abstractions rather than changing the facts.

Give the answer a clear structure

Use **situation, responsibility, actions, result and reflection** as a flexible structure. Start with the answer to the actual question. Give only enough background to make your choice understandable, then spend the detail on what you decided and did. End with what happened and what you changed afterward. This is an organizing aid, not a script or a timing exercise.

For each substantial story, retain these facts:

- **Problem and stakes:** The affected user, observable pain, and why action mattered.

- **Your boundary:** Your role, authority, collaborators and the work you did not own.

- **Decision:** The alternatives, uncertainty, tradeoff, and strongest objection.

- **Actions:** Concrete implementation and interpersonal steps in a clear sequence.

- **Outcome:** What changed, how you know, what remained unsuccessful, and others' contributions.

- **Reflection:** What you would change and a later example of applying that lesson, if one exists.

Maintain a short opening and deeper material behind it. A technology question calls for the mechanism; a collaboration question calls for the people and decisions. The same event may support both, but the emphasis should change.

An illustrative answer shape

The following is fictional and demonstrates specificity. It is not one of your stories.

“Several teams were waiting on our group whenever they needed a test environment. I owned the provisioning service; another engineer owned its interface. I initially proposed a broad rewrite, but conversations with users showed that setup failures were their immediate blocker. I compared repairing that path with replacing the service, then chose the smaller change because we could validate it sooner.

“One team worried about losing custom settings. I worked with its engineer on a pilot, documented which settings we could preserve, and kept a route back to the old flow. The pilot

completed the agreed setup tasks without our manual intervention, but one unusual configuration remained unsupported. I would report that limit alongside the improvement. I learned to validate the workflow with users before expanding the architecture."

The useful features are a defined customer, limited personal scope, a changed assumption, a decision with a cost, a stakeholder concern, evidence and an unresolved limitation. In your answer, replace every event with a real one and include measurements only when you can substantiate them.

Misleading signals and how to prevent them

These are interpretation risks to check while editing stories, not secret rejection triggers.

How an answer can sound	Evidence that makes your meaning clearer
"We built it" hides your contribution	Name what you personally decided, implemented or coordinated, then credit collaborators.
"I handled everything" hides teamwork	Explain dependencies, advice that changed your mind and what others owned.
"I needed agreement from everyone" suggests stalled decisions	Identify the decision owner and distinguish consultation from required approval.
"I just did it" suggests avoidable surprises	Explain authority, affected people, communication and risk limits.
"I am very direct" suggests disregard for others	Describe the actual conversation, listening and how the recipient could act on it.

How an answer can sound	Evidence that makes your meaning clearer
“The migration shipped” leaves adoption uncertain	Account for customer transitions, support, exceptions and remaining maintenance.
“The metric improved” leaves causality uncertain	Give the baseline, population, period, countermeasure and uncertainty.
“I fixed it by working all night” leaves a recurring dependency	Explain the subsequent engineering or prioritization change.

The public mock specifically highlights personal attribution, answering the requested question, and explaining relationships when discussing feedback to leadership. Preserve those distinctions without adding events that never happened. Mock feedback.

A commercial guide with input from a former Netflix product leader also emphasizes decision ownership. That is useful triangulation, but its PM examples and claims about evaluation should not override your recruiter brief or be treated as engineering policy. Culture interview guide.

Questions to ask the hiring manager

Select questions whose answers would help you understand the role. Follow the conversation rather than working through the entire list.

- Which team and problem area within this posting is this position for, and what made hiring for it important now?

- What would a successful L5 contribution look like during the first six months? Where should this person decide independently, and where is broader coordination essential?

- Who are the main users, and what is their most painful workflow today? How does the team learn about problems that do not become support tickets?

- How do you judge whether a platform investment is helping? Can you describe a tool or initiative you changed because adoption or results disappointed you?

- Where do partner teams most often resist migration, and what has helped them move safely?

- Can you give an example of an engineer changing the team's direction through disagreement or customer evidence?

- What does useful feedback look like between you and a senior engineer? How are expectations made clear when priorities change or an experiment fails?

- How is responsibility split among development, operation, support and product work? What currently consumes more maintenance effort than you would like?

- Which languages and systems would I use most? How much breadth across UI, APIs and data is needed immediately, and where can someone grow into it?

- Is there an area of my experience you would like me to clarify before we finish?

Listen for concrete examples, decision boundaries, customer evidence and candid discussion of current difficulties. This conversation also helps you assess whether the work and environment suit you.

Story coverage for preparation

Build a small set of distinct events, not a memorized answer for every possible question. Two substantial projects can supply several stories if each story has its own decision and outcome. Add separate events where those projects do not cover an important behaviour.

Story to find	Main question coverage
A project with clear personal ownership and a difficult technical choice	Q03, Q04, Q06
A developer workflow improvement with real customer feedback	Q02, Q05, Q06
A migration or change involving other teams	Q08, Q09, Q13
An independent decision with incomplete information	Q07, Q14
A meaningful mistake, failed bet or incident	Q10, Q12, Q16
Critical feedback received and a later behavioural change	Q10, Q18
Difficult feedback given respectfully	Q08, Q11
Helping someone contribute or choosing a wider benefit	Q14, Q15

The story bank holds the working facts and drafts. The conversation session provides a route for developing them aloud. Both begin empty of personal examples; authoring this material does not imply any practice has been completed.

Sources and how to use them

All links were checked for this research on 13 October 2026. The role's matching requisition ID was verified in the supplied page and Netflix's Workday listing. Their wording can change independently.

- **Primary role evidence:** Supplied Developer Platform role and JR32657 on Workday. These define the role, not interview questions.

- **Primary culture evidence:** Current memo and revision announcement, 24 June 2024. Use the current memo over old slide decks or simplified summaries.

- **Official candidate advice with a different level scope:** New-graduate guidance. Used only for its invitation to discuss culture and projects.

- **Primary engineering context:** Full Cycle Developers, 17 May 2018 and Cassandra Data Movement, 18 May 2026. Examples of engineering thinking; neither establishes this team's current architecture.

- **Observed practice and coaching:** Public L5 mock. Read as an example of follow-up and feedback, not an actual hiring decision.

- **Candidate reports:** Frontend community, January 2026 accounts, SWE report dated February 2025, SRE thread, Warsaw full-stack report, and L5 Ads report. Self-reported and not independently verified; useful for themes and variation.

- **Secondary synthesis:** IGotAnOffer culture guide, updated 27 September 2026. Commercial advice used as a cross-check, not as authority for Netflix policy.

Your recruiter brief, supplied in this conversation on 13 October 2026, is the strongest evidence for this particular meeting's agenda.

Sources and scope

Hiring-manager preparation was added on 13 October 2026. Chapter 13 has its own research and source notes, based on the supplied Developer Platform role, recruiter guidance, official Netflix material, a public mock and candidate reports. Its questions and assessment guidance are original preparation material. The technical-course scope below remains the history of chapters 1-12.

Original scope research checked on 30 September 2026. Reading edition revised on 4 October 2026; Go specification, heap, sync, and memory-model references rechecked for the revised examples. The exercises and learning sequence are original preparation material; they are not a leaked or verified Netflix question bank.

What informed the preparation

Your plan.md is the primary scope input. It supplies the core algorithm sequence and practical topics, particularly LRU, TTL, rate limiting, event processing, and iterative requirements. Your clarification establishes an L5 live interview with another senior engineer. No interview date or team-specific preparation guide was supplied, and no chapter requires a particular study duration.

Internet searches covered Netflix technical-screen descriptions, CodeSignal's own documentation, recommendation-system sources, and Go references. Search results contained conflicting commercial guides and secondhand reports. Those reports were not treated as proof of what you will be asked. The live format and level come from you, not from those reports.

Primary sources used

- CodeSignal live interviewing supports the distinction between an interviewer-led session and an assessment. It does not specify Netflix's content or evaluation rubric.

- CodeSignal practice overview supports practising the IDE and question formats. Your invitation remains the source for session-specific constraints.

- CodeSignal language support lists platform language support visually. We do not infer a particular Go runtime version from it; verify that in your actual environment.

- Netflix recommendation overview supplies high-level product context, not a coding interview syllabus.

- Google candidate generation overview explains content-based and collaborative retrieval. The toy ranking formulas and tasks here are our own.

- Go container/heap and sort.Search support heap interface and binary-search API details. Current source documentation was also fetched through Context7 using /golang/go.

- Go sync, memory model, and time support synchronization and clock discussion.

- Go specification and strings, bytes, runes and characters support language behavior.

- Go test command and race detector explain local validation tools. Race detection checks executed paths and does not prove absence of all races.

Coverage of the original plan

Two Sum, anagrams, windows, and array products appear in lesson 2. Three Sum, intervals, binary search, top k, and kth-element selection share lesson 3's ordering tools. Tree traversal, BST validation, islands, course dependencies, and graph cloning appear in lesson 4. LRU, weighted capacity, TTL, combined expiration, and concurrency are lessons 6 and 7. The two rate limiters are lesson 8. Time-based storage, deduplication, counters, and scheduling are lesson 9. Streaming statistics and recommendation tasks are lesson 10. Filesystem, transactions, pub/sub, and playback-service composition are chapter 11 and its worked program design.

The reading edition now includes 65 Go blocks extracted into one compiler-checked example package. Algorithms and the LRU, weighted cache, TTL map, transaction stack, and combined metadata-cache baseline are assembled implementations. Filesystem resolution, pub/sub publishing, and scheduler claiming/draining are explicitly labelled cores with their surrounding preconditions. They are not presented as complete implementations of every optional service method. The final ranking case composes the earlier comparator with filtering, deduplication, and aggregation. The conversation edition remains separate.

A further connection

For kth largest in a stream, use a size-k min heap over all seen values; duplicates count as separate observations unless specified otherwise. For a static array, sorting is a simple baseline and quickselect offers expected linear time, but its worst case and pivot handling require care. For this first-round route, reliable heap reasoning is the higher-priority transferable skill.

Limits of the material

This course provides focused preparation, not exhaustive coverage of every possible interview problem. Deep distributed systems, advanced ML training, and a full Go language course are outside this first coding-round scope. The book can be read independently without an editor, running tests, or an AI. Spoken implementations develop reasoning but are not evidence that code has compiled.

Repository and conversation access

The book is independent of a particular ChatGPT interface. For the repository workflow, an agent must actually retrieve the relevant files. Do not infer that remote read access includes permission or tools to update progress. Official ChatGPT Voice documentation describes conversational turn-taking and task permissions. Official workspace connection guidance distinguishes resource access from write actions and recommends checking them separately. These pages do not verify this reader's account configuration.